
ETS: Energy-Guided Test-Time Scaling for Training-Free RL Alignment

Xiuyu Li^{*1} Jinkai Zhang^{*1} Mingyang Yi¹ Yu Li¹ Longqiang Wang¹ Yue Wang² Ju Fan¹

Abstract

Reinforcement Learning (RL) post-training alignment for language models is effective, but also costly and unstable in practice, owing to its complicated training process. To address this, we propose a training-free inference method to sample directly from the optimal RL policy. The transition probability applied to Masked Language Modeling (MLM) consists of a reference policy model and an energy term. Based on this, our algorithm, Energy-Guided Test-Time Scaling (ETS), estimates the key energy term via online Monte Carlo, with a provable convergence rate. Moreover, to ensure practical efficiency, ETS leverages modern acceleration frameworks alongside tailored importance sampling estimators, substantially reducing inference latency while provably preserving sampling quality. Experiments on MLM (including autoregressive models and diffusion language models) across reasoning, coding, and science benchmarks show that our ETS consistently improves generation quality, validating its effectiveness and design. The code is available at <https://github.com/sheriyuo/ETS>.

1. Introduction

Recently, Reinforcement Learning (RL) has become a central paradigm for aligning large language models (LLMs) with human intent (Guo et al., 2025; Ouyang et al., 2022; Li et al., 2026b) in a post-training stage (Kumar et al., 2025). Technically, in the RL stage, the LLM is recognized as a policy model (Sutton et al., 1998), which interacts with a reward model (Schulman et al., 2017; Rafailov et al., 2024; Shao et al., 2024). Despite their empirical success in reasoning (Guo et al., 2025), image generation (Team et al., 2025), natural language tasks (Hou et al., 2024; Luo et al., 2026)

and agentic workflows (Li et al., 2026a), this paradigm suffers from many fundamental limitations: it requires costly reward modeling (Ouyang et al., 2022; Bai et al., 2022); large-scale human preference data (Rafailov et al., 2024; Zhang et al., 2025b); exhibits unstable training dynamics (Zheng et al., 2025a; Deng et al., 2026) and sensitivity to hyperparameters (Shao et al., 2024); and must be rerun whenever the reward design changes (Huan et al., 2025), leading to substantial computational and human costs.

Theoretically, the existing KL-regularized RL objective of LLM admits a closed-form solution (Shao et al., 2024; Rafailov et al., 2024). However, existing training-based approaches (Zhu et al., 2025) rely on iterative gradient-based optimization to approximate this optimal solution, rather than directly exploiting its structure. This raises a fundamental question: *if the optimal RL distribution is known in closed form, can we sample from it directly at inference time, without any additional training?* By doing so, all the aforementioned issues brought by post-training are resolved.

Inspired by this, we explore inference-time sampling directly from the optimal RL distribution. Theoretically, we show that for the general masked language modeling (MLM) framework (Austin et al., 2021), which includes autoregressive models (ARMs) (Achiam et al., 2023; Yang et al., 2025) and diffusion language models (DLMs) (Nie et al., 2025; Ou et al., 2025), the optimal transition kernel decomposes into two factors: (1) The original transition kernel provided by the reference model. (2) An energy term (Lu et al., 2023; Xu et al., 2025) given by a conditional expectation of exponentiated rewards. This formulation enables *training-free* sampling from the optimal RL policy model.

With the transition formulation, we design Energy-Guided Test-Time Scaling (ETS) to sample from the target optimal distribution without any training process. Moreover, we prove the convergence rate measured by total variation distance (Duchi, 2016) of our method. Notably, our method is conceptually analogous to Monte Carlo Tree Search (MCTS) (Yao et al., 2023) and can be naturally interpreted as a form of Test-Time Scaling (TTS) (Zhang et al., 2025a): additional computation at inference is used to explore multiple candidate continuations and guide the generation toward outcomes with high reward (He et al., 2025). Similar to other TTS methods, the central challenge

^{*}Equal contribution ¹Renmin University of China, Beijing, China ²Zhongguancun Academy, Beijing, China. Correspondence to: Mingyang Yi <ymingyang@ruc.edu.cn>.

of our method is how to perform this exploration efficiently without incurring prohibitive latency. Here, the bottleneck is the energy term in the transition kernel. To resolve this, we design efficient estimators for the energy term and corresponding sampling algorithms tailored to different model families.

To accelerate decoding, for ARMs, we adopt a small proposal model (Yang et al., 2025); for DLMs, we use FastdLLM (Wu et al., 2025) with KV caching and parallel decoding. When combined with importance sampling (Tokdar & Kass, 2010), both lightweight proposal models produce Monte Carlo unbiased energy estimators with substantially reduced latency. Theoretically, we prove these estimators yield samples converging to the target distribution.

Empirically, we evaluate ETS across both ARMs and DLMs on reasoning, coding, and science benchmarks (Hendrycks et al., 2021; Rein et al., 2024; Chen, 2021). Our method consistently improves generation quality over standard inference and test-time scaling baselines (Kang et al., 2025; Karan & Du, 2025), surpassing post-trained RL policy without any training. Comprehensive ablations validate our sampling procedures, reward design, and acceleration strategies, demonstrating favorable quality-latency trade-offs.

2. Related Work

Reinforcement Learning (RL). RL was originally proposed as a framework for trial-and-error learning in sequential decision-making and optimal control (Sutton et al., 1998; Mnih et al., 2015; Watkins & Dayan, 1992; Schulman et al., 2017). Recently, RL has been brought into the LLM post-training, most notably through RLHF and its variants, to align LLM behavior with human preferences and to improve reasoning capabilities (Ouyang et al., 2022; Guo et al., 2025; Zhang et al., 2025b; Su et al., 2025). Common RL-based post-training algorithms for LLMs include PPO, DPO, and GRPO (Schulman et al., 2017; Rafailov et al., 2024; Guo et al., 2025). However, these methods suffer from high retraining costs, reliance on expensive human preference data, strong sensitivity to hyperparameters, and lack of well-defined rewards (Casper et al., 2023; Rafailov et al., 2024).

Test-Time Scaling (TTS). TTS improves model performance by allocating additional computation at inference time rather than updating parameters (Snell et al., 2025). Representative TTS techniques include Best-of-N (Nakano et al., 2021), Self-Consistency (Wang et al., 2022; Shafayat et al., 2025), Beam Search (Sutskever et al., 2014), and Monte Carlo Tree Search (MCTS) (Yao et al., 2023; Chang et al., 2025). Beam Search maintains multiple candidate sequences during decoding and approximates maximum a posteriori generation via width-limited pruning, while MCTS enhances reasoning accuracy by sampling diverse solution

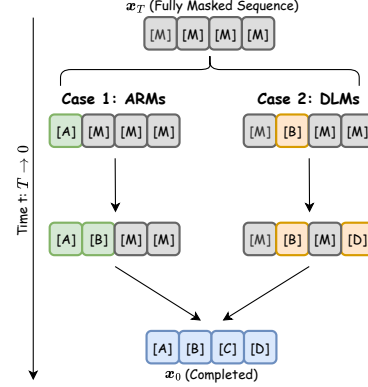


Figure 1. Unified MLM framework. Generation is modeled as a backward Markov chain from x_T to x_0 . Case 1 shows the fixed left-to-right decoding path of ARMs; Case 2 illustrates the flexible, non-sequential unmasking of DLMs.

paths and selecting the most consistent or highest-scoring outcome. Similar to this paper, Power Sampling (Karan & Du, 2025) and Quest (Faria et al., 2024) also sample from RL post-training target distribution by Metropolis–Hastings (MH) algorithm (Chib & Greenberg, 1995), leading to slow generation due to repeated recursive LLM queries. Moreover, (Balashankar et al., 2025; Fei et al., 2025) propose to fit TTS with the target distribution of RL, but their gains heavily rely on training-based RL components rather than offering a general, training-free sampling solution.

Recently, (Dang et al., 2025; Uehara et al., 2024; Singhal et al., 2025) proposed a similar framework to sample from the RLHF post-trained objective by intervening in the generation of trajectories. However, their methods are limited to diffusion models (Ho et al., 2020; Austin et al., 2021; Liu et al., 2025b; Chen et al., 2026). Besides, unlike our method, their methods lack a theoretical convergence rate, and cannot be applied without a verifiable reward function. More importantly, our methods are more efficient than theirs due to our acceleration methods.

Acceleration. The goal of inference-time acceleration is to reduce the latency while keeping the model’s output distribution essentially unchanged (Zhou et al., 2024; Song et al., 2026). Techniques such as speculative decoding (Leviathan et al., 2023; Li et al., 2025; Hu et al., 2025), quantization (Zheng et al., 2025b), pruning (Ma et al., 2023), and sparse attention (Liu et al., 2025a) are widely used in LLM deployment for this purpose. Since the energy term in our transition kernel can be estimated by a Monte Carlo method, these acceleration methods can be integrated with our method.

3. Background

Masked Language Modeling (MLM) Framework. We consider an MLM framework that subsumes both ARMs

and DLMs (Austin et al., 2021). Let the vocabulary be denoted by $\mathcal{V}$, and $\mathbf{x}_0 = \{x_0^1, \dots, x_0^{d_x}\} \in \mathcal{V}^{d_x}$ be a sequence of length d_x with x_0^i as the i -th token. We introduce a special token $[\text{mask}] \in \mathcal{V}$ and define a series of sequences of partially masked states $\{\mathbf{x}_t\}_{t=0}^T$ with

$$x_t^i = \begin{cases} x_0^i, & i \notin M_t; \\ [\text{mask}], & i \in M_t. \end{cases} \quad (1)$$

Here $M_t \subseteq \{1, \dots, d_x\}$ is a subset denoting the set of masked positions at step t , satisfying $M_s \subset M_t$ for $s < t$ and $M_T = \{1, \dots, d_x\}$.

In this paper, we only consider the decoding stage of MLM (as shown in Figure 1). That is, the model progressively generates $\mathbf{x}_0$ starting from $\mathbf{x}_T$ (T is given in advance). Thus, the mask index decides the decoding order. For ARMs, the mask index $M_t = \{d_x - t + 1, \dots, d_x\}$ corresponds to left-to-right generation. For DLMs, M_t is a random subset of $\{1, \dots, d_x\}$ depending on the logits of predicted tokens.

The decoding process is a backward Markov process with transition kernel

$$p(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y}) = \prod_{i=s+1}^t p(\mathbf{x}_{i-1} | \mathbf{x}_i, \mathbf{y}) \quad (2)$$

for any $s < t$, where $\mathbf{y}$ is a prompt or query. For MLM, the conditional probability $p(\mathbf{x}_{i-1} | \mathbf{x}_i, \mathbf{y})$ is modeled by a function call of LLM. For ARMs, the transition $p(\mathbf{x}_{i-1} | \mathbf{x}_i, \mathbf{y})$ corresponds to the standard next-token prediction (Achiam et al., 2023). For DLMs, the $p(\mathbf{x}_{i-1} | \mathbf{x}_i, \mathbf{y})$ is obtained by modeling the conditional distribution $p(\mathbf{x}_0 | \mathbf{x}_i, \mathbf{y})$ to predict $\mathbf{x}_0$, and then remask it into $\mathbf{x}_{i-1}$. In this paper, for DLMs, we consider the decoding strategy in (Nie et al., 2025) by unmasking the tokens with the top- K predicted probabilities in each decoding step, i.e.,

$$|M_i| - |M_{i-1}| = K, \quad K = \frac{d_x}{T}. \quad (3)$$

RL Method. We consider a general question-answering setting. Given a query $\mathbf{y}$, the goal is to generate a response $\mathbf{x}_0 = \{x_0^1, \dots, x_0^{d_x}\} \in \mathcal{V}^{d_x}$. The query-response pair $(\mathbf{y}, \mathbf{x}_0)$ is evaluated by a reward function $r(\mathbf{y}, \mathbf{x}_0) \in \mathcal{R}$, where $\mathcal{R}$ may be discrete or continuous.

In this setting, the answering policy is modeled by a LLM $p(\mathbf{x}_0 | \mathbf{y})$. Then, the RL-based post-training methods aim to solve the following KL-regularized optimization problem:

$$\max_{p(\mathbf{x}_0 | \mathbf{y})} \mathbb{E}_{p(\mathbf{x}_0 | \mathbf{y})} [r(\mathbf{y}, \mathbf{x}_0) - \lambda D_{\text{KL}}(p(\mathbf{x}_0 | \mathbf{y}) \| p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}))], \quad (4)$$

where $p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y})$ denotes a fixed reference model and $\lambda > 0$ is a constant. As can be seen, the goal of RLHF is to maximize the expected reward under the trained policy model, while maintaining its inherent ability by regularizing it to stay close to the reference model.

Proposition 1. (Rafailov et al., 2024) *The RLHF objective (4) has a closed-form solution*

$$p(\mathbf{x}_0 | \mathbf{y}) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right)}{C}, \quad (5)$$

where $C = \sum_{\mathbf{x}_0} p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp(r(\mathbf{y}, \mathbf{x}_0)/\lambda)$ is a normalizing constant.

4. Sample From the Target Distribution

In this section, we present a training-free method to sample from the RL target distribution. Our method has theoretically guaranteed convergence and is applicable to the general MLMs.

4.1. Energy Reweighted Backward Transition

Under the unified MLM framework in Section 3, the generation process follows a backward Markov chain $\mathbf{x}_T \rightarrow \mathbf{x}_{T-1} \rightarrow \dots \rightarrow \mathbf{x}_0$. Therefore, sampling from the optimal distribution (5) reduces to specifying the backward transition kernel $p(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y})$ under distribution (5), for each pair of steps $s < t$.

The following proposition derives an explicit expression for this transition by combining the reference model's transition with the expectation of the exponential reward.

Proposition 2. *For MLM, for any given query $\mathbf{y}$ and response $\mathbf{x}_0$, we have*

$$\begin{aligned} p(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y}) &= \frac{p(\mathbf{x}_t | \mathbf{x}_s) p(\mathbf{x}_s | \mathbf{x}_0)}{p_{\text{ref}}(\mathbf{x}_t | \mathbf{x}_0)} \\ &= \frac{p_{\text{ref}}(\mathbf{x}_s | \mathbf{y}) \mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) \right]}{p_{\text{ref}}(\mathbf{x}_t | \mathbf{y}) \mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_t)} \left[\exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) \right]} \\ &\propto p_{\text{ref}}(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y}) \mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) \right]. \end{aligned} \quad (6)$$

Concretely, (6) expresses the target backward transition as a product of two components. (1) The transition kernel $p_{\text{ref}}(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y})$ under the reference model; (2) An energy term $\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} [\exp(r(\mathbf{y}, \mathbf{x}_0)/\lambda)]$, which evaluates how likely the state $\mathbf{x}_s$ is to lead to high-reward completions under the reference model.

Please notice that the intermediate ratio form in (6) arises from rewriting the RL-optimal joint distribution over the backward trajectory and then isolating the dependence on $\mathbf{x}_s$ relative to $\mathbf{x}_t$; factors independent of the candidate choice of $\mathbf{x}_s$ can be absorbed into the normalization constant.

Remark 1. Notably, (Dang et al., 2025; Uehara et al., 2024; Singhal et al., 2025) propose a similar transition kernel as (6) under the continuous-time diffusion model by

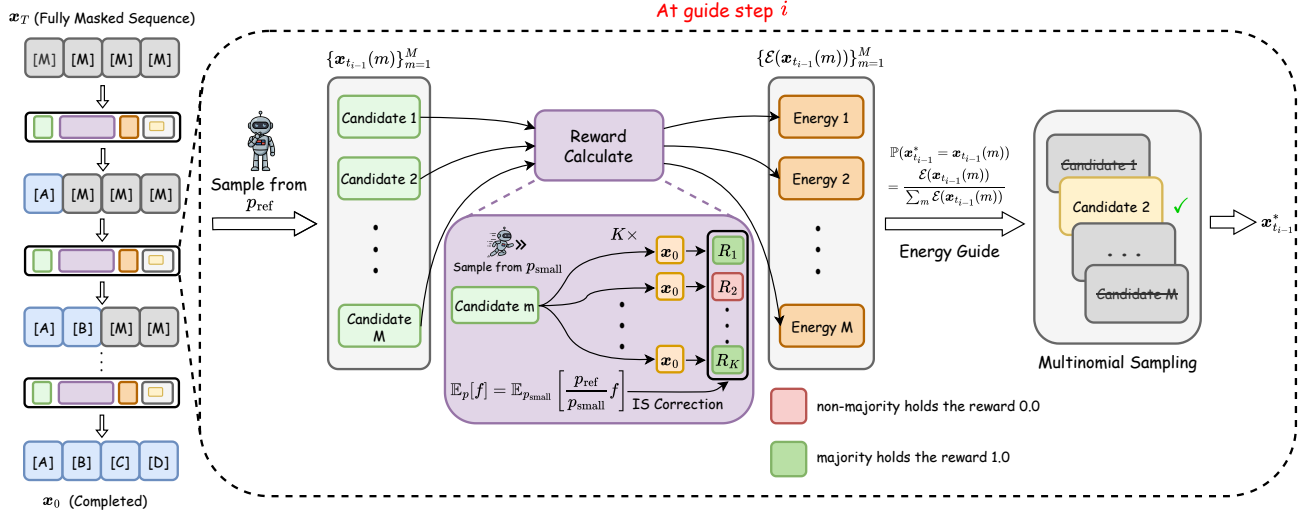


Figure 2. Overview of Energy-Guided Test-Time Scaling (ETS). ETS performs iterative guidance on the unified MLM framework. At each guidance step (zoomed-in, right), the algorithm evaluates M candidates. Their associated energy $\mathcal{E}$ is estimated via Monte Carlo method using K independent completions of the corresponding candidate. We utilize an aligned lightweight model p_{small} with Importance Sampling (IS) correction to accelerate energy estimation while maintaining theoretical consistency with the target optimal distribution.

Algorithm 1 Energy-Guided Test-Time Scaling (ETS): sampling directly from the RL optimal policy.

Input: Reference model p_{ref} , Guidance steps I , Timesteps $\{t_0 = 0, t_1 = \frac{T}{I}, \dots, t_I = T\}$, Initialization $\mathbf{x}_T$, Query $\mathbf{y}$, Reward function r , Candidates number M .

Output: Sample $\mathbf{x}_0$.

- 1: **for** $i = I, \dots, 1$ **do**
- 2: Sampling $\{\mathbf{x}_{t_{i-1}}(m)\}_{m=1}^M$ from $p_{\text{ref}}(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})$
- 3: Computing $w_m = \frac{\hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))}{\sum_{m=1}^M \hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))}$ under (8).
- 4: Selecting $m^* \sim \text{Multinomial}(M, w_1, \dots, w_M)$
- 5: Taking $\mathbf{x}_{t_{i-1}} = \mathbf{x}_{t_{i-1}}(m^*)$
- 6: **end for**

continuous-time control theory (Åström, 2012). Compared with their results, our result (6) can be applied to general discrete MLM, and is directly derived from the properties of MLM. (please refer to Appendix A.2 for details)

4.2. Monte Carlo Method

Leveraging the transition kernel (6), we can sample directly from the optimal distribution (5). To implement this, we should estimate the energy term

$$\mathcal{E}(\mathbf{y}, \mathbf{x}_s) = \mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right], \quad (7)$$

since the transition $p_{\text{ref}}(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y})$ can be directly evaluated. Thus, a straightforward way to estimate $\mathcal{E}(\mathbf{y}, \mathbf{x}_s)$ is

the Monte Carlo estimation (Metropolis & Ulam, 1949)

$$\mathcal{E}(\mathbf{y}, \mathbf{x}_s) \approx \hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_s) = \frac{1}{K} \sum_{k=1}^K \exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0(k))}{\lambda} \right), \quad (8)$$

$$\{\mathbf{x}_0(k)\}_{k=1}^K \stackrel{i.i.d.}{\sim} p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s).$$

With this, we propose Algorithm 1 to approximately sample from the optimal distribution via self-normalizing importance sampling (Hammersley, 2013; Cardoso et al., 2022). Intuitively, while the global normalizing constant is intractable, self-normalizing the energies of a sampled batch recovers the relative probabilities among candidates. Sampling from this reweighted batch is equivalent to sampling from the optimal distribution restricted to these candidates.

In Algorithm 1, we guide the generation process with the energy term in steps $t_1, \dots, t_I$ with $1 \leq I \leq T$. Increasing the number of guidance steps I allows for more frequent re-alignment with the optimal policy, thereby improving the quality of the generated samples. However, this comes at the expense of higher latency, as each guidance step necessitates additional Monte Carlo estimations of the energy term. Notably, by selecting $I = 1$ and $\lambda \rightarrow 0$, the Algorithm 1 becomes Best-of-N (Zhang et al., 2025a) since the normalized weights $w_m \propto \mathcal{E}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))$ degenerate to the largest one within the finite candidate set.

Notably, for ETS, ARMs equipped with *batching* and *early-stop* mechanisms decode $M \times K$ sequences in parallel and often terminate early, yielding lower actual per-token cost and fewer generated tokens, as shown in Section 6.2, Appendix B.1 and B.2. Similar to Best-of-N and Beam Search (Wolf et al., 2020), parallel generation algorithms like ETS

better utilize GPU memory and FLOPs, offering greater speed advantages compared to sequential generation methods like Power Sampling (Karan & Du, 2025).

Compared with the existing methods (Dang et al., 2025; Uehara et al., 2024; Singhal et al., 2025), our method demonstrates advantages in applicability, theory, and efficiency. (1) The previous works primarily focus on continuous-time diffusion models, while our unified framework seamlessly covers both DLMs and ARMs. (2) As formally established in Proposition 3, we provide rigorous theoretical guarantees proving that our sampling distribution converges to the optimal target in total variation distance. (3) We address the high latency of existing TTS using the acceleration method with bias correction in Section 5, significantly reducing inference time while maintaining performance. (4) Unlike methods that rely on verifiable reward functions or require fine-tuning value functions, our approach is entirely training-free and operates without such constraints.

4.3. Sampling Error Analysis

Algorithm 1 is exact if we can evaluate the energy term in Proposition 2 exactly and draw infinitely many candidates. In practice, however, we approximate the conditional expectation by $\hat{\mathcal{E}}$ (Section 4.2) and use a finite candidate set of size M to perform the reweighting-and-resampling step. These two approximations induce a deviation between the induced output distribution $q(\mathbf{x}_0 | \mathbf{y})$ of Algorithm 1 and the target distribution $p(\mathbf{x}_0 | \mathbf{y})$.

To quantify this deviation, we measure the discrepancy in total variation distance¹ (Duchi, 2016) and provide an explicit bound that captures how estimation error and finite- M sampling error accumulate across the I guidance steps. The following proposition formalizes this guarantee.

Proposition 3. *Suppose that for any given query $\mathbf{y}$ and response $\mathbf{x}_s$, we have $|\mathcal{E}(\mathbf{y}, \mathbf{x}_s) - \hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_s)| \leq \epsilon$ for some $\epsilon > 0$. Then for the $\mathbf{x}_0 \sim q(\mathbf{x}_0 | \mathbf{y})$ as in Algorithm 1, we have*

$$\begin{aligned} D_{\text{TV}}(q(\mathbf{x}_0 | \mathbf{y}) \| p(\mathbf{x}_0 | \mathbf{y})) \\ \leq I \left(\frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} \right) + I\epsilon = \tilde{O} \left(\frac{I}{\sqrt{M}} + I\epsilon \right), \end{aligned} \quad (9)$$

where

$$h(\epsilon, M, \lambda, D) = \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{2}{M} \log \left(\frac{2}{\epsilon} \right)}. \quad (10)$$

The upper bound reflects the fluctuation induced by finite candidate sampling, scaling as $1/M$, ϵ and modulated by the constant C, D, λ . This indicates that the total variation

¹The total variation distance between distribution p and q is $D_{\text{TV}}(p \| q) = \frac{1}{2} \int |p(x) - q(x)| dx$.

distance grows linearly with I , and can be suppressed by increasing M or reducing ϵ . In particular, when $M \rightarrow \infty$ and $\epsilon \rightarrow 0$, the algorithm converges to the target distribution.

Remark 2. *Notably, our bound on the total variation distance is linearly increasing with the guidance steps I . This is similar to the results in the continuous diffusion model (Yi et al., 2023; Chen et al., 2023; Yi et al., 2024; Wang et al., 2025), which indicate that the total error accumulates in each denoising step. However, this does not mean that taking $I = 1$ (Best-of- N decoding) presents the best sampling error. This is because we assume the guidance error ϵ is identical in each guidance step (from $\mathbf{x}_{t_i}$ to estimate $\mathbf{x}_0$). However, in practice, the guidance error ϵ may vary across different $\mathbf{x}_t$, which may result in a different sampling error bound compared to ours. We leave such a more delicate bound as future work.*

4.4. Reward Design

Estimating the energy term $\mathcal{E}(\mathbf{y}, \mathbf{x}_s)$ in (7) requires a verifiable reward function $r(\mathbf{y}, \mathbf{x})$, such as correctness against ground truth or human preference scores. However, direct access to the ground-truth reward during inference is impractical (Kang et al., 2025). While prior work trains models to approximate it (Ouyang et al., 2022; Zhang et al., 2025b), this conflicts with our *training-free* objective. We therefore design a training-free *proxy reward* $r(\mathbf{y}, \mathbf{x})$ that operates without ground-truth, yet induces a reward distribution closely approximating the true one.

We adopt a self-consistency reward derived from the model itself: for each candidate, we sample K completions, extract their final answers, and assign reward 1 to answers matching the majority vote and 0 otherwise. This mechanism favors answers that are statistically consistent across multiple trials, approximating the behavior of a ground-truth reward.

Moreover, we examine various training-free reward candidates, including token-level confidence (logits) (Kadavath et al., 2022), predictive entropy (Kuhn et al., 2023), self-certainty (Kang et al., 2025), and self-consistency (Wang et al., 2022; Shafayat et al., 2025). As shown in Appendix B.4 and D.2, among these uncertainty-based metrics, self-consistency yields a reward distribution closest to the ground-truth reward, providing a tight upper bound on performance attainable with true reward supervision.

5. Acceleration Method

In this section, we introduce inference-time acceleration techniques for both ARMs and DLMs. The Monte Carlo estimation of the energy term in (8) incurs substantial computational overhead. To enable practical deployment, we develop efficient estimators that preserve theoretical guarantees while significantly reducing latencies.

Algorithm 2 ETS-IS (Importance Sampling): accelerate ETS by drawing samples from a lightweight proposal.

Input: Reference model p_{ref} , Small model p_{small} , Guidance steps I , Timesteps $\{t_0 = 0, t_1 = \frac{T}{I}, \dots, t_I = T\}$, Initialization $\mathbf{x}_T$, Query $\mathbf{y}$, Reward function r , Candidates number M .

Output: Sample $\mathbf{x}_0$.

- 1: **for** $i = I, \dots, 1$ **do**
- 2: Sampling $\{\mathbf{x}_{t_{i-1}}(m)\}_{m=1}^M$ from $p_{\text{ref}}(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})$
- 3: Computing $\hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))$ under importance sampling method (11).
- 4: Computing $w_m = \frac{\hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))}{\sum_{m=1}^M \hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))}$
- 5: Selecting $m^* \sim \text{Multinomial}(M, w_1, \dots, w_m)$
- 6: Taking $\mathbf{x}_{t_{i-1}} = \mathbf{x}_{t_{i-1}}(m^*)$
- 7: **end for**

5.1. General Method

Our key observation is that the Monte Carlo estimator only depends on samples drawn from p_{ref} . Therefore, by replacing samples from p_{ref} with those from a computationally cheaper proposal model, we can significantly reduce the latency while preserving the correctness of the estimator.

By importance sampling (Tokdar & Kass, 2010), we know that for another small proposal model p_{small} close to p_{ref} , we have

$$\begin{aligned}
 \mathcal{E}(\mathbf{y}, \mathbf{x}_s) &= \mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right] \\
 &= \mathbb{E}_{p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}{p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right] \\
 &\approx \frac{1}{K} \sum_{k=1}^K \frac{p_{\text{ref}}(\mathbf{x}_0(k) | \mathbf{y}, \mathbf{x}_s)}{p_{\text{small}}(\mathbf{x}_0(k) | \mathbf{y}, \mathbf{x}_s)} \exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0(k))}{\lambda} \right) \\
 &= \hat{\mathbb{E}}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right] \\
 &= \hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_s),
 \end{aligned} \tag{11}$$

where $\{\mathbf{x}_0(k)\}_{k=1}^K \stackrel{i.i.d.}{\sim} p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)$. Thus, we can also estimate the energy term by sampling from a cheaper p_{small} . Our acceleration method to approximate the target energy term is summarized in Algorithm 2. Moreover, we have the following proposition to analyze the approximation error in this regime.

Proposition 4. For any given query $\mathbf{y}$, noisy response $\mathbf{x}_s$ and bounded $r(\mathbf{y}, \mathbf{x}_0) \in [-D, D]$, if there exists a constant $L > 0$ such that $l(\mathbf{x}_0) \leq L$ for all $\mathbf{x}_0$, then with probability at least $1 - \delta$ we have

$$\left| \mathcal{E}(\mathbf{y}, \mathbf{x}_s) - \hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_s) \right| \leq L e^{\frac{D}{\lambda}} \sqrt{\frac{\log(2/\delta)}{2K}}, \tag{12}$$

where

$$l(\mathbf{x}_0) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}{p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}. \tag{13}$$

This proposition shows the trade-off between latency (using p_{small}) and estimation accuracy: a larger K reduces the error but increases inference time, while a poorly matched proposal (large L) may require more samples to achieve the same precision. Then, by combining this proposition with Proposition 3, we obtain the following theorem.

Theorem 1. For any given query $\mathbf{y}$, noisy response $\mathbf{x}_s$, bounded $r(\mathbf{y}, \mathbf{x}_0) \in [-D, D]$ and $\mathbf{x}_0 \sim q(\mathbf{x}_0 | \mathbf{y})$ as in Algorithm 2, if there exists a constant $L > 0$ such that $l(\mathbf{x}_0) \leq L$ for all $\mathbf{x}_0$, then we have

$$\begin{aligned}
 D_{\text{TV}}(q(\mathbf{x}_0 | \mathbf{y}) \parallel p(\mathbf{x}_0 | \mathbf{y})) \\
 \leq I \left(\frac{2\epsilon + g(\epsilon, M, K, L, \lambda, D)}{C - \epsilon - g(\epsilon, M, K, L, \lambda, D)} \right) + 4I \exp \left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}} \right) \\
 = \tilde{\mathcal{O}} \left(\frac{I}{\sqrt{M}} + \frac{I}{\sqrt{K}} \right),
 \end{aligned} \tag{14}$$

where

$$\begin{aligned}
 \epsilon &= \tilde{\mathcal{O}} \left(\frac{1}{\sqrt{K}} \right), \quad l(\mathbf{x}_0) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}{p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}, \\
 g(\epsilon, M, K, L, \lambda, D) &= \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{4K\epsilon^2}{ML^2 e^{\frac{2D}{\lambda}}}} \\
 &= \tilde{\mathcal{O}} \left(\frac{1}{\sqrt{M}} \right).
 \end{aligned} \tag{15}$$

By choosing K sufficiently large, Algorithm 2 can recover the same asymptotic accuracy as Algorithm 1.

5.2. Acceleration for ARMs

For ARMs, we adopt a *scale-aligned* lightweight model as the proposal distribution p_{small} to accelerate energy estimation. Specifically, we use Qwen3 lightweight models that share the same tokenization with the reference model, which are naturally suitable as p_{small} due to their similarity. Substituting this choice into Algorithm 2 yields the ETS-IS for ARMs, providing a favorable trade-off between computational efficiency and estimator variance.

We note that alternative approaches like quantization and speculative decoding face practical limitations. Quantized models do not provide operator-level speed advantages at our computational scale, while the speculative decoding method EAGLE-3 (Li et al., 2025) does not support our batching techniques (Appendix B.2) due to tree attention and integration with vLLM (Kwon et al., 2023) inference frameworks. Our ETS-IS approach with distilled models balances implementation feasibility with performance benefits, and we leave the integration with the other acceleration methods as future work (Su et al., 2026; Tang et al., 2026).

5.3. Acceleration for DLMs

Unlike ARMs, no small DLM aligned with LLaDA-8B-Instruct is available to serve as p_{small} . Therefore, We adopt Fast-dLLM (Wu et al., 2025) as the acceleration scheme for DLMs. Fast-dLLM accelerates inference through two key mechanisms: (1) A block-wise approximate KV cache that reuses similar activations across steps in bidirectional attention. (2) Confidence-aware parallel decoding, which selects high-confidence tokens via a global threshold to increase throughput while preserving quality. We treat the distribution induced by Fast-dLLM as the proposal distribution p_{small} and apply importance sampling to correct the energy estimation, yielding ETS-IS for DLMs.

6. Experiments

6.1. Experimental Setup

Evaluation. We evaluate our methods on a standard suite of reasoning benchmarks covering mathematics (MATH500, GSM8K), coding (HumanEval), and STEM (GPQA). All methods and baselines are evaluated in a single-shot (pass@1) setting, i.e., based on a single final response.

- **MATH500:** The MATH dataset (Hendrycks et al., 2021) consists of competition-level mathematics problems spanning seven categories, containing 12,500 problems with 7,500 training problems and 5,000 test problems. MATH500 is a randomly selected subset of 500 test problems standardized by OpenAI.
- **GSM8K:** GSM8K (Cobbe et al., 2021) is a collection of grade-school-level math word problems designed to evaluate multi-step numerical reasoning. The dataset consists of 8,500 problems in total, with 7,473 training problems and 1,319 test problems.
- **HumanEval:** HumanEval (Chen, 2021) is a benchmark of 164 handwritten programming problems covering algorithms, reasoning, mathematics, and language comprehension. Each problem is associated with an average of 7.7 unit tests, where solving the problem corresponds to passing all unit tests.
- **GPQA:** GPQA (Rein et al., 2024) is a dataset of multiple-choice science questions, designed to require advanced domain reasoning. We evaluate on the GPQA Diamond subset, which consists of 198 questions representing the highest-quality portion of the dataset.

Models. To demonstrate the universality of our method across different generative paradigms, we conduct experiments on both ARMs and DLMs. For ARMs, we use Qwen3-1.7B and Qwen3-8B (Yang et al., 2025) in the non-thinking mode as backbone models; For DLMs, we use

LLaDA-8B-Instruct (Nie et al., 2025) as the backbone model (LLaDA is only available in 8B).

For all benchmarks, we use lm-eval (Gao et al., 2024) to evaluate both ARMs and DLMs. We refer readers to Appendix C.2 for more detailed hyperparameters.

Baseline. The compared baseline methods are categorized into three groups: (1) The standard inference method Base; (2) The TTS methods Beam Search (Wolf et al., 2020), Best-of-N and Power Sampling (Karan & Du, 2025); (3) The RL baseline from Verl (Sheng et al., 2024) and LLaDA-1.5 (Zhu et al., 2025). All of our experiments are reproduced on $4 \times \text{H20-3e-141GB}$ GPUs and match the existing SOTA (Karan & Du, 2025; Zhu et al., 2025). We refer readers to Appendix C.1 for more detailed settings.²

6.2. Main Results

We present our main results in Table 1, which reports accuracy and average latency across tasks. Three key observations emerge:

1. Across different model families, our ETS achieves *substantial* and *universal* boosts in accuracy across different reasoning tasks, consistently matching and even surpassing the performance of the post-trained RL policy (GRPO) without any parameter updates.
2. The acceleration method speeds up the sampling process with slight performance degradation.
3. The latency of our ETS is comparable to the standard TTS methods, e.g., Best-of-N, while achieving significantly better accuracy.

All these results verify the effectiveness and efficiency of ETS methods. Notably, due to the inherently limited parallel decoding speed of LLaDA, its TTS methods incur substantially higher latency than ARMs. Even under this constraint, our acceleration method provides a meaningful reduction in latency while preserving competitive performance.

6.3. Ablation Study

We ablate key design choices of our method: total sample count $M \times K$, guidance steps I , and the trade-off between latency and accuracy. All other hyperparameters follow the settings in Section 6.2. We present results on Qwen3-8B as a representative ARM; additional ablations on DLM-specific design choices, reward design, temperature sensitivity, and generation length are provided in Appendix D.

²We do not use ETS-IS for Qwen3-1.7B model since its inference latency is low enough.

Table 1. ETS (ours) outperforms other TTS methods and GRPO across model families and tasks. We benchmark the performance of ETS on MATH500, GSM8K, GPQA and HumanEval. Reported accuracies are the best achieved within constrained latency (time and accuracy averaged across datasets), with the best values in each setting bolded.

Methods	MATH500	GSM8K	GPQA	HumanEval	Avg	Time
<i>Qwen3-1.7B</i>						
Base	37.2	69.29	29.80	39.63	43.98	1×
Beam Search	45.8	70.81	26.77	46.34	47.43	1.59×
Best-of-N	58.8	81.58	27.78	39.63	51.70	3.47×
Power Sampling	50.6	-	26.30	44.51	-	40.9×
ETS (ours)	60.4	81.88	29.80	46.95	54.76	7.18×
GRPO	52.8	71.57	25.25	42.68	48.08	1×
<i>Qwen3-8B</i>						
Base	60.0	89.39	34.34	58.54	60.57	1×
Beam Search	61.2	89.46	32.83	69.51	63.25	1.99×
Best-of-N	65.2	94.09	38.89	67.07	66.31	5.16×
Power Sampling	70.8	-	28.80	71.91	-	41.8×
ETS (ours)	72.4	94.24	38.38	71.34	69.34	7.35×
ETS-IS (ours)	66.2	91.81	38.89	68.90	66.45	5.26×
GRPO	69.2	90.98	37.88	65.85	65.98	1×
<i>LLaDA-8B-Instruct</i>						
Base	42.2	80.36	28.28	35.98	46.71	1×
Beam Search	42.8	82.71	27.78	38.41	47.93	17.6×
Best-of-N	44.2	85.82	28.28	36.59	48.90	17.4×
ETS (ours)	45.6	85.97	32.83	39.63	51.01	30.6×
ETS-IS (ours)	43.8	85.82	28.79	39.02	49.36	16.1×
VRPO	40.2	82.03	23.74	39.63	46.40	1×

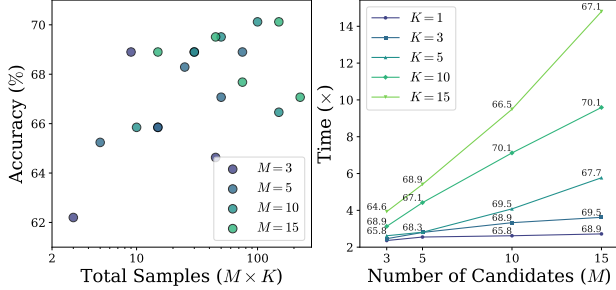


Figure 3. Effect of total samples on ETS. We ablate the total samples with Qwen3-8B and plot HumanEval accuracies (left) with corresponding latencies (right) for various sample counts.

Total Samples. Figure 3 demonstrates that appropriate $M \times K$ settings yield better results while retaining low latency. In most cases, both accuracy and latency increase as $M \times K$ grows. This is consistent with our theoretical results (Proposition 3). Moreover, we observe that scaling M is more effective for improving accuracy than scaling K . According to our empirical observation, selecting $K = 3$ strikes a good balance between accuracy and efficiency.

Guidance Steps. In our Algorithm 2, the number of guidance steps is determined by I , where a larger I leads to stronger guidance but higher latency. To see its influence, we vary I in Figure 4, which shows that scaling the guidance steps I yields proportional improvements in accuracy alongside increased latency. Owing to this trade-off, we suggest $I = 4$ or 8 in practice to balance latency and ac-

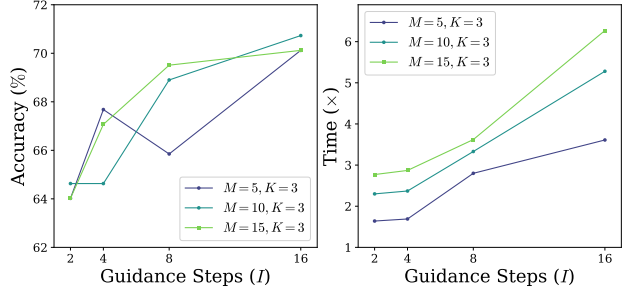


Figure 4. Effect of guidance steps on ETS. We evaluate Qwen3-8B on HumanEval (left) with corresponding latencies (right) are reported under various guidance steps.

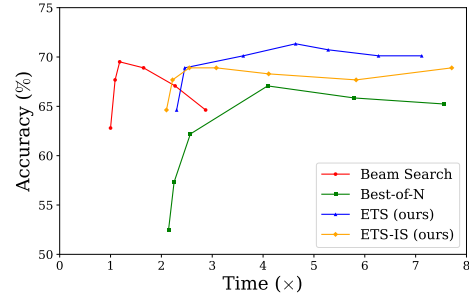


Figure 5. Comparisons between TTS methods. We ablate different latencies and plot corresponding HumanEval accuracies with Qwen3-8B, for various training-free TTS methods.

curacy. Moreover, the empirical observations verify that guiding the intermediate decoding steps indeed improves the performance of the reasoning process.

Time and Accuracy. We compare ETS with the other TTS methods regarding the trade-off between latency and accuracy. As shown in Figure 5, ETS achieves superior accuracy at comparable latency to other TTS methods. This advantage is pronounced as the computational budget increases. Interestingly, we find that the performance of baseline TTS methods declines significantly with more samples due to verification noise, failing to utilize additional computation effectively (Chow et al., 2024). However, unlike these methods, which naively scale the number of candidates M , ETS concurrently scales the prediction range K to produce a smaller set of higher-quality reasoning paths, leading to more robust verification. Moreover, we summarize a practical guideline for our ETS as follows: for constrained budgets, ETS-IS offers efficient initial gains; for larger budgets, standard ETS achieves higher peak performance.

7. Conclusion

In this paper, we present ETS, a training-free method for sampling directly from the optimal RL policy in masked language models. By formulating the sampling process as

a Monte Carlo estimation of an energy term, ETS ensures provable convergence to the target distribution and is further accelerated through modern inference frameworks coupled with tailored importance sampling estimators. Experiments across autoregressive and diffusion models on reasoning and coding benchmarks show that ETS exceeds post-trained RL policy, offering an efficient alternative to costly post-training alignment. This work establishes a practical, training-free pathway toward reward-aware generation.

Acknowledgments

This work was supported by NSF of China Projects (No.62506365).

Impact Statement

This work demonstrates that ETS enables training-free alignment by directly sampling from the optimal reinforcement learning policy, offering a cost-effective and stable alternative to traditional RL post-training methods. Our framework establishes a foundation for future research on TTS and efficient inference in generative models. This can democratize alignment techniques for resource-constrained settings and reduce the environmental footprint of LLM deployment.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. Preprint arXiv:2303.08774, 2023.
- Åström, K. J. *Introduction to stochastic control theory*. Courier Corporation, 2012.
- Austin, J., Johnson, D. D., Ho, J., Tarlow, D., and Van Den Berg, R. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 2021.
- Bai, Y., Kadavath, S., Kundu, S., et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. Preprint arXiv:2204.05862, 2022.
- Balashankar, A., Sun, Z., Berant, J., Eisenstein, J., Collins, M., Hutter, A., Lee, J., Nagpal, C., Prost, F., Sinha, A., Suresh, A. T., and Beirami, A. Infalign: Inference-aware language model alignment. In *International Conference on Machine Learning*, 2025.
- Cardoso, G., Samsonov, S., Thin, A., Moulines, E., and Olsson, J. Br-snis: bias reduced self-normalized importance sampling. *Advances in Neural Information Processing Systems*, 2022.
- Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., et al. Open problems and fundamental limitations of reinforcement learning from human feedback. Preprint arXiv:2307.15217, 2023.
- Chang, K., Shi, Y., Wang, C., Zhou, H., Hu, C., Liu, X., Luo, Y., Ge, Y., Xiao, T., and Zhu, J. Step-level verifier-guided hybrid test-time scaling for large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- Chen, M. Evaluating large language models trained on code. Preprint arXiv:2107.03374, 2021.
- Chen, S., Chewi, S., Li, J., Li, Y., Salim, A., and Zhang, A. Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions. In *International Conference on Learning Representations*, 2023.
- Chen, Y., Guo, X., Shi, Z., Song, Z., and Zhang, J. T2VWorldBench: A benchmark for evaluating world knowledge in text-to-video generation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2026.
- Chib, S. and Greenberg, E. Understanding the metropolis-hastings algorithm. *The american statistician*, 1995.
- Chow, Y., Tennenholtz, G., Gur, I., Zhuang, V., Dai, B., Thiagarajan, S., Boutilier, C., Agarwal, R., Kumar, A., and Faust, A. Inference-aware fine-tuning for best-of-n sampling in large language models. Preprint arXiv:2412.15287, 2024.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. Preprint arXiv:2110.14168, 2021.
- Dang, M., Han, J., Xu, M., Xu, K., Srivastava, A., and Ermon, S. Inference-time scaling of diffusion language models with particle gibbs sampling. Preprint arXiv:2507.08390, 2025.
- Deng, W., Li, Y., Gong, B., Ren, Y., Thrampoulidis, C., and Li, X. On group relative policy optimization collapse in agent search: The lazy likelihood-displacement. In *ICLR 2026 Workshop on Lifelong Agents: Learning, Aligning, Evolving*, 2026.
- Duchi, J. Lecture notes for statistics 311/electrical engineering 377. 2016. URL https://web.stanford.edu/class/stats311/OldSyllabi/full_notes.pdf.

- Faria, G., Agrawal, S., Farinhas, A., Rei, R., de Souza, J., and Martins, A. Quest: Quality-aware metropolis-hastings sampling for machine translation. *Advances in Neural Information Processing Systems*, 2024.
- Fei, Y., Razeghi, Y., and Singh, S. Nudging: Inference-time alignment of llms via guided decoding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.
- Fu, W., Gao, J., Shen, X., Zhu, C., Mei, Z., He, C., Xu, S., Wei, G., Mei, J., Wang, J., et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. Preprint arXiv:2505.24298, 2025.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. The language model evaluation harness, 2024. URL <https://zenodo.org/records/12608602>.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. Preprint arXiv:2501.12948, 2025.
- Hammersley, J. *Monte carlo methods*. Springer Science & Business Media, 2013.
- He, J., Fan, J., Jiang, B., Houine, I., Roth, D., and Ribeiro, A. Self-give: Associative thinking from limited structured knowledge for enhanced large language model reasoning. Preprint arXiv:2505.15062, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *Advances in Neural Information Processing Systems*, 2021.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 2020.
- Hou, Z., Niu, Y., Du, Z., Zhang, X., Liu, X., et al. Chatglm-rlhf: Practices of aligning large language models with human feedback. Preprint arXiv:2404.00934, 2024.
- Hu, Z., Meng, J., Akhauri, Y., Abdelfattah, M. S., Seo, J.-s., Zhang, Z., and Gupta, U. Flashdlm: Accelerating diffusion language model inference via efficient kv caching and guided diffusion. Preprint arXiv:2505.21467, 2025.
- Huan, M., Li, Y., Zheng, T., Xu, X., Kim, S., Du, M., Poovendran, R., Neubig, G., and Yue, X. Does math reasoning improve general llm capabilities? understanding transferability of llm reasoning. Preprint arXiv:2507.00432, 2025.
- Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., et al. Language models (mostly) know what they know. Preprint arXiv:2207.05221, 2022.
- Kang, Z., Zhao, X., and Song, D. Scalable best-of-n selection for large language models via self-certainty. Preprint arXiv:2502.18581, 2025.
- Karan, A. and Du, Y. Reasoning with sampling: Your base model is smarter than you think. Preprint arXiv:2510.14901, 2025.
- Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. Preprint arXiv:2302.09664, 2023.
- Kumar, K., Ashraf, T., Thawakar, O., Anwer, R. M., Cholakkal, H., Shah, M., Yang, M.-H., Torr, P. H., Khan, F. S., and Khan, S. Llm post-training: A deep dive into reasoning large language models. Preprint arXiv:2502.21321, 2025.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Symposium on Operating Systems Principles*, 2023.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-3: Scaling up inference acceleration of large language models via training-time test. Preprint arXiv:2503.01840, 2025.
- Li, Y., Yi, M., Li, X., Fan, J., Jiang, F., Chen, B., Li, P., Song, J., and Zhang, T. Reasoning and tool-use compete in agentic rl: from quantifying interference to disentangled tuning. Preprint arXiv:2602.00994, 2026a.
- Li, Z., Yi, M., Wang, Y., Cui, S., and Liu, Y. Towards a theoretical understanding to the generalization of rlhf. Preprint arXiv:2601.16403, 2026b.
- Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. Deepseek-v3. 2: Pushing the frontier of open large language models. Preprint arXiv:2512.02556, 2025a.
- Liu, C., Zhang, J., Wang, S., Fan, W., and Li, Q. Score-based generative diffusion models for social recommendations. *IEEE Transactions on Knowledge and Data Engineering*, 2025b.

- Lu, C., Chen, H., Chen, J., Su, H., Li, C., and Zhu, J. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In *International Conference on Machine Learning*, 2023.
- Luo, S., Zhang, X., Hu, Y., Zhang, B., Wang, K., Su, J., Sun, M., Liang, L., and Zhang, J. Sparse-rl: Breaking the memory wall in llm reinforcement learning via stable sparse rollouts. Preprint arXiv:2601.10079, 2026.
- Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 2023.
- Metropolis, N. and Ulam, S. The monte carlo method. *Journal of the American statistical association*, 1949.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 2015.
- Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. Webgpt: Browser-assisted question-answering with human feedback. Preprint arXiv:2112.09332, 2021.
- Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models. Preprint arXiv:2502.09992, 2025.
- Ou, J., Nie, S., Xue, K., Zhu, F., Sun, J., Li, Z., and Li, C. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. In *International Conference on Learning Representations*, 2025.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 2022.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., and Ermon, S. Direct preference optimization: Your language model is secretly a reward model. Preprint arXiv:2305.18290, 2024.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. Preprint arXiv:1707.06347, 2017.
- Shafayat, S., Tajwar, F., Salakhutdinov, R., Schneider, J., and Zanette, A. Can large reasoning models self-train? Preprint arXiv:2505.21444, 2025.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. Preprint arXiv:2402.03300, 2024.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. Preprint arXiv:2409.19256, 2024.
- Singhal, R., Horvitz, Z., Teehan, R., Ren, M., Yu, Z., McKeown, K., and Ranganath, R. A general framework for inference-time scaling and steering of diffusion models. Preprint arXiv:2501.06848, 2025.
- Snell, C. V., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Song, Z., Xue, J., Zhang, J., and Zhang, L. Sublinear time quantum algorithm for attention approximation. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Su, J., Hu, Y., Li, C., Chen, H., Li, J., Ma, L., and Zhang, J. Tablecache: Primary foreign key guided kv cache precomputation for low latency text-to-sql. Preprint arXiv:2601.08743, 2026.
- Su, X., Wang, Y., Zhu, J., Yi, M., Xu, F., Ma, Z., and Liu, Y. Reveal the mystery of dpo: The connection between dpo and rl algorithms. Preprint arXiv:2502.03095, 2025.
- Sutskever, I., Vinyals, O., and Le, Q. V. Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, 27, 2014.
- Sutton, R. S., Barto, A. G., et al. *Reinforcement learning: An introduction*. 1998.
- Tang, Z., Verma, A., Zhou, Z., Wu, Z., Prakash, A., Rus, D., and Low, B. K. H. Minedraft: A framework for batch parallel speculative decoding. Preprint arXiv:2603.18016, 2026.
- Team, M. L., Ma, H., Tan, H., Huang, J., Wu, J., He, J.-Y., Gao, L., Xiao, S., Wei, X., Ma, X., et al. Longcat-image technical report. Preprint arXiv:2512.07584, 2025.
- Tokdar, S. T. and Kass, R. E. Importance sampling: a review. *Wiley Interdisciplinary Reviews: Computational Statistics*, 2010.
- Uehara, M., Zhao, Y., Black, K., Hajiramezanali, E., Scalia, G., Diamant, N. L., Tseng, A. M., Biancalani, T., and Levine, S. Fine-tuning of continuous-time diffusion models as entropy-regularized control. Preprint arXiv:2402.15194, 2024.

- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. Preprint arXiv:2203.11171, 2022.
- Wang, Z., Yi, M., Xue, S., Li, Z., Liu, M., Qin, B., and Ma, Z.-M. Improved diffusion-based generative model with better adversarial robustness. In *International Conference on Learning Representations*, 2025.
- Watkins, C. J. and Dayan, P. Q-learning. *Machine learning*, 1992.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020.
- Wu, C., Zhang, H., Xue, S., Liu, Z., Diao, S., Zhu, L., Luo, P., Han, S., and Xie, E. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. Preprint arXiv:2505.22618, 2025.
- Xu, M., Geffner, T., Kreis, K., Nie, W., Xu, Y., Leskovec, J., Ermon, S., and Vahdat, A. Energy-based diffusion language models for text generation. In *International Conference on Learning Representations*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. Preprint arXiv:2505.09388, 2025.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 2023.
- Yi, M., Sun, J., and Li, Z. On the generalization of diffusion model. Preprint arXiv:2305.14712, 2023.
- Yi, M., Matabuena, M., and Wang, R. Denoising data with measurement error using a reproducing kernel-based diffusion model. Preprint arXiv:2501.00212, 2024.
- Zhang, Q., Lyu, F., Sun, Z., Wang, L., Zhang, W., Hua, W., Wu, H., Guo, Z., Wang, Y., Muennighoff, N., et al. A survey on test-time scaling in large language models: What, how, where, and how well? Preprint arXiv:2503.24235, 2025a.
- Zhang, Y. and Math-AI, T. American invitational mathematics examination (aime) 2024, 2024.
- Zhang, Y., Fan, M., Fan, J., Yi, M., Luo, Y., Tan, J., and Li, G. Reward-sql: Boosting text-to-sql via step-wise reasoning and process-supervised rewards. Preprint arXiv:2505.04671, 2025b.
- Zheng, C., Dang, K., Yu, B., Li, M., Jiang, H., Lin, J., Liu, Y., Yang, A., Zhou, J., and Lin, J. Stabilizing reinforcement learning with llms: Formulation and practices. Preprint arXiv:2512.01374, 2025a.
- Zheng, X., Li, Y., Chu, H., Feng, Y., Ma, X., Luo, J., Guo, J., Qin, H., Magno, M., and Liu, X. An empirical study of qwen3 quantization. Preprint arXiv:2505.02214, 2025b.
- Zhou, Z., Ning, X., Hong, K., Fu, T., Xu, J., Li, S., Lou, Y., Wang, L., Yuan, Z., Li, X., et al. A survey on efficient inference for large language models. Preprint arXiv:2404.14294, 2024.
- Zhu, F., Wang, R., Nie, S., Zhang, X., Wu, C., Hu, J., Zhou, J., Chen, J., Lin, Y., Wen, J.-R., et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. Preprint arXiv:2505.19223, 2025.

A. Proofs

A.1. Proof of RL Optimal Distribution

Proposition 1. (Rafailov et al., 2024) *The RLHF objective (4) has a closed-form solution*

$$p(\mathbf{x}_0 | \mathbf{y}) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right)}{C}, \quad (5)$$

where $C = \sum_{\mathbf{x}_0} p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp(r(\mathbf{y}, \mathbf{x}_0)/\lambda)$ is a normalizing constant.

Proof. We consider the KL-regularized RL objective

$$\max_{p(\mathbf{x}_0 | \mathbf{y})} \mathbb{E}_{p(\mathbf{x}_0 | \mathbf{y})} [r(\mathbf{y}, \mathbf{x}_0) - \lambda D_{\text{KL}}(p(\mathbf{x}_0 | \mathbf{y}) \| p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}))]. \quad (16)$$

Expanding the expectation and the KL divergence, the objective can be written as

$$\mathcal{J}(p) = \sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) r(\mathbf{y}, \mathbf{x}_0) - \lambda \sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) \log \frac{p(\mathbf{x}_0 | \mathbf{y})}{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y})}. \quad (17)$$

We maximize $\mathcal{J}(p)$ subject to the normalization constraint $\sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) = 1$. Introducing a Lagrange multiplier α , we obtain the Lagrangian

$$\begin{aligned} \mathcal{L}(p, \alpha) &= \sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) r(\mathbf{y}, \mathbf{x}_0) - \lambda \sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) \log p(\mathbf{x}_0 | \mathbf{y}) \\ &\quad + \lambda \sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) \log p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) + \alpha \left(\sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) - 1 \right). \end{aligned} \quad (18)$$

Taking the functional derivative with respect to $p(\mathbf{x}_0 | \mathbf{y})$ and setting it to zero yields

$$\frac{\partial \mathcal{L}}{\partial p(\mathbf{x}_0 | \mathbf{y})} = r(\mathbf{y}, \mathbf{x}_0) - \lambda (\log p(\mathbf{x}_0 | \mathbf{y}) + 1) + \lambda \log p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) + \alpha = 0. \quad (19)$$

Solving for $p(\mathbf{x}_0 | \mathbf{y})$ gives

$$p(\mathbf{x}_0 | \mathbf{y}) = \exp\left(\frac{\alpha}{\lambda} - 1\right) p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right)}{C}, \quad (20)$$

where $C = \frac{1}{\exp(\frac{\alpha}{\lambda} - 1)}$ is a normalization constant.

Imposing the constraint $\sum_{\mathbf{x}_0} p(\mathbf{x}_0 | \mathbf{y}) = 1$, we obtain

$$1 = \frac{\sum_{\mathbf{x}_0} p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right)}{C}, \quad (21)$$

which implies

$$C = \sum_{\mathbf{x}_0} p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}) \exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right). \quad (22)$$

Then we prove our conclusion. □

A.2. Proof of Backward Transition

Proposition 2. *For both ARMs and DLMs, for any given query $\mathbf{y}$ and response $\mathbf{x}_0$, we have*

$$\begin{aligned} p(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y}) &= \frac{p(\mathbf{x}_t | \mathbf{x}_s) p(\mathbf{x}_s | \mathbf{x}_0) p_{\text{ref}}(\mathbf{x}_s | \mathbf{y})}{p_{\text{ref}}(\mathbf{x}_t | \mathbf{x}_0) p_{\text{ref}}(\mathbf{x}_t | \mathbf{y})} \frac{\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) \right]}{\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_t)} \left[\exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) \right]} \\ &\propto p_{\text{ref}}(\mathbf{x}_s | \mathbf{x}_t, \mathbf{y}) \mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)} \left[\exp\left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda}\right) \right]. \end{aligned} \quad (23)$$

Proof. Let U_s to denote the index of unmasked tokens in $\mathbf{x}_s$ with complement M_s . By construction of the diffusion process, the clean sequence $\mathbf{x}_0$ agrees with $\mathbf{x}_s$ on the unmasked positions, i.e., $\mathbf{x}_0^{U_s} = \mathbf{x}_s^{U_s}$. Consequently, conditioning on $\mathbf{x}_s$ is equivalent to conditioning on the fixed values of $\mathbf{x}_0^{U_s}$ together with a noise pattern on M_s , i.e.,

$$p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_s) = p_{\text{ref}}(\mathbf{x}_0^{M_s} \mid \mathbf{y}, \mathbf{x}_0^{U_s} = \mathbf{x}_s^{U_s}) = p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_0^{U_s}), \quad (24)$$

Similarly, we note that $p_{\text{ref}}(\mathbf{x}_0^{U_s} \mid \mathbf{y}) = p_{\text{ref}}(\mathbf{x}_s^{U_s} \mid \mathbf{y}) = p_{\text{ref}}(\mathbf{x}_s \mid \mathbf{y})$.

Due to property of total probability and the Markov property of $\mathbf{x}_t$,

$$\begin{aligned} p(\mathbf{x}_s \mid \mathbf{y}, \mathbf{x}_t) &= \sum_{\mathbf{x}_0} p(\mathbf{x}_s, \mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_t) \\ &= \sum_{\mathbf{x}_0} p(\mathbf{x}_s \mid \mathbf{x}_0, \mathbf{x}_t) p(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_t) \\ &= \sum_{\mathbf{x}_0^{M_s}} \sum_{\mathbf{x}_0^{U_s}} \frac{p(\mathbf{x}_0, \mathbf{x}_s, \mathbf{x}_t)}{p(\mathbf{x}_0, \mathbf{x}_t)} \frac{p(\mathbf{x}_0, \mathbf{x}_t \mid \mathbf{y})}{p(\mathbf{x}_t \mid \mathbf{y})} \\ &= \sum_{\mathbf{x}_0^{M_s}} \sum_{\mathbf{x}_0^{U_s}} \frac{p(\mathbf{x}_t \mid \mathbf{x}_s) p(\mathbf{x}_s \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{x}_0)} \frac{p(\mathbf{x}_t \mid \mathbf{x}_0) p(\mathbf{x}_0 \mid \mathbf{y})}{p(\mathbf{x}_t \mid \mathbf{y})} \\ &= \frac{p(\mathbf{x}_t \mid \mathbf{x}_s) p(\mathbf{x}_s \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{y})} \sum_{\mathbf{x}_0^{M_s}} p_{\text{ref}}(\mathbf{x}_0^{U_s}, \mathbf{x}_0^{M_s} \mid \mathbf{y}) \frac{\exp(r(\mathbf{y}, \mathbf{x}_0))}{C} \\ &= \frac{p(\mathbf{x}_t \mid \mathbf{x}_s) p(\mathbf{x}_s \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{y})} p_{\text{ref}}(\mathbf{x}_0^{U_s} \mid \mathbf{y}) \sum_{\mathbf{x}_0^{M_s}} p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_0^{U_s}) \frac{\exp(r(\mathbf{y}, \mathbf{x}_0))}{C} \\ &= \frac{p(\mathbf{x}_t \mid \mathbf{x}_s) p(\mathbf{x}_s \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{y})} p_{\text{ref}}(\mathbf{x}_0^{U_s} \mid \mathbf{y}) \frac{\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_0^{U_s})} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right]}{C} \\ &= \frac{p(\mathbf{x}_t \mid \mathbf{x}_s) p(\mathbf{x}_s \mid \mathbf{x}_0)}{p(\mathbf{x}_t \mid \mathbf{y})} p_{\text{ref}}(\mathbf{x}_s \mid \mathbf{y}) \frac{\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_s)} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right]}{C}. \end{aligned} \quad (25)$$

On the other hand,

$$\begin{aligned} p(\mathbf{x}_t \mid \mathbf{y}) &= \sum_{\mathbf{x}_0^{U_t}} \sum_{\mathbf{x}_0^{M_t}} p(\mathbf{x}_t, \mathbf{x}_0 \mid \mathbf{y}) \\ &= p(\mathbf{x}_t \mid \mathbf{x}_0) \sum_{\mathbf{x}_0^{M_t}} \frac{p_{\text{ref}}(\mathbf{x}_0^{U_t}, \mathbf{x}_0^{M_t} \mid \mathbf{y}) \exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right)}{C} \\ &= p(\mathbf{x}_t \mid \mathbf{x}_0) p_{\text{ref}}(\mathbf{x}_0^{U_t} \mid \mathbf{y}) \sum_{\mathbf{x}_0^{M_t}} \frac{p_{\text{ref}}(\mathbf{x}_0^{M_t} \mid \mathbf{y}, \mathbf{x}_0^{U_t}) \exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right)}{C} \\ &= p(\mathbf{x}_t \mid \mathbf{x}_0) p_{\text{ref}}(\mathbf{x}_0^{U_t} \mid \mathbf{y}) \frac{\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_0^{U_t})} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right]}{C} \\ &= p(\mathbf{x}_t \mid \mathbf{x}_0) p_{\text{ref}}(\mathbf{x}_t \mid \mathbf{y}) \frac{\mathbb{E}_{p_{\text{ref}}(\mathbf{x}_0 \mid \mathbf{y}, \mathbf{x}_t)} \left[\exp \left(\frac{r(\mathbf{y}, \mathbf{x}_0)}{\lambda} \right) \right]}{C}. \end{aligned} \quad (26)$$

By plugging this into the above equation, we prove our conclusion. $\square$

A.3. Proof of Total Variance Distance

Proposition 3. Suppose that for any given query $\mathbf{y}$ and response $\mathbf{x}_t$, we have $|\mathcal{E}(\mathbf{y}, \mathbf{x}_t) - \widehat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_t)| \leq \epsilon$ for some $\epsilon > 0$. Then for the $\mathbf{x}_0 \sim q(\mathbf{x}_0 \mid \mathbf{y})$ as in Algorithm 1, we have

$$D_{\text{TV}}(q(\mathbf{x}_0 \mid \mathbf{y}) \parallel p(\mathbf{x}_0 \mid \mathbf{y})) \leq I \left(\frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} \right) + I\epsilon = \tilde{O} \left(\frac{I}{\sqrt{M}} + I\epsilon \right), \quad (27)$$

where

$$h(\epsilon, M, \lambda, D) = \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{2}{M} \log \left(\frac{2}{\epsilon} \right)}. \quad (28)$$

Proof. For any given $i \in \{1, \dots, I\}$, we construct a distribution $\tilde{q}$ such that $\tilde{q}(\mathbf{x}_{t_i} \mid \mathbf{y}) = q(\mathbf{x}_{t_i} \mid \mathbf{y})$ while $\tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) = p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i})$. Then by the triangle inequality we have

$$D_{\text{TV}}(p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) \parallel q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})) \leq D_{\text{TV}}(p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) \parallel \tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})) + D_{\text{TV}}(\tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) \parallel q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})). \quad (29)$$

by the triangle's inequality. Due to the definition of total variation distance, we have

$$\begin{aligned} D_{\text{TV}}(p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}), \tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})) &= \frac{1}{2} \sum_{\mathbf{x}_{t_{i-1}}} |p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) - \tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})| \\ &= \frac{1}{2} \sum_{\mathbf{x}_{t_{i-1}}} \left| \sum_{\mathbf{x}_{t_i}} p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) p(\mathbf{x}_{t_i} \mid \mathbf{y}) - \sum_{\mathbf{x}_{t_i}} \tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) \tilde{q}(\mathbf{x}_{t_i} \mid \mathbf{y}) \right| \\ &= \frac{1}{2} \sum_{\mathbf{x}_{t_{i-1}}} \left| \sum_{\mathbf{x}_{t_i}} p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) [p(\mathbf{x}_{t_i} \mid \mathbf{y}) - \tilde{q}(\mathbf{x}_{t_i} \mid \mathbf{y})] \right| \\ &\leq \frac{1}{2} \sum_{\mathbf{x}_{t_i}} \sum_{\mathbf{x}_{t_{i-1}}} p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) |p(\mathbf{x}_{t_i} \mid \mathbf{y}) - \tilde{q}(\mathbf{x}_{t_i} \mid \mathbf{y})| \\ &= D_{\text{TV}}(p(\mathbf{x}_{t_i} \mid \mathbf{y}) \parallel \tilde{q}(\mathbf{x}_{t_i} \mid \mathbf{y})) \\ &= D_{\text{TV}}(p(\mathbf{x}_{t_i} \mid \mathbf{y}) \parallel q(\mathbf{x}_{t_i} \mid \mathbf{y})). \end{aligned} \quad (30)$$

On the other hand, we note that

$$\begin{aligned} &D_{\text{TV}}(\tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) \parallel q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})) \\ &= \sup_{f: \|f\|_{\infty} \leq 1} \left| \sum_{\mathbf{x}_{t_{i-1}}} f(\mathbf{x}_{t_{i-1}}) \tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) - \sum_{\mathbf{x}_{t_{i-1}}} f(\mathbf{x}_{t_{i-1}}) q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) \right| \\ &= \sup_{f: \|f\|_{\infty} \leq 1} \left| \sum_{\mathbf{x}_{t_{i-1}}} f(\mathbf{x}_{t_{i-1}}) \sum_{\mathbf{x}_{t_i}} \tilde{q}(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) q(\mathbf{x}_{t_i} \mid \mathbf{y}) - \sum_{\mathbf{x}_{t_{i-1}}} f(\mathbf{x}_{t_{i-1}}) \sum_{\mathbf{x}_{t_i}} q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) q(\mathbf{x}_{t_i} \mid \mathbf{y}) \right| \\ &= \sup_{f: \|f\|_{\infty} \leq 1} \left| \sum_{\mathbf{x}_{t_i}} q(\mathbf{x}_{t_i} \mid \mathbf{y}) \sum_{\mathbf{x}_{t_{i-1}}} f(\mathbf{x}_{t_{i-1}}) p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) - \sum_{\mathbf{x}_{t_i}} q(\mathbf{x}_{t_i} \mid \mathbf{y}) \sum_{\mathbf{x}_{t_{i-1}}} f(\mathbf{x}_{t_{i-1}}) q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i}) \right| \\ &= \sup_{f: \|f\|_{\infty} \leq 1} \left| \sum_{\mathbf{x}_{t_i}} q(\mathbf{x}_{t_i} \mid \mathbf{y}) \left(\mathbb{E}_{p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i})} [f(\mathbf{x}_{t_{i-1}})] - \mathbb{E}_{q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i})} [f(\mathbf{x}_{t_{i-1}})] \right) \right| \\ &\leq \sup_{f: \|f\|_{\infty} \leq 1} \sum_{\mathbf{x}_{t_i}} q(\mathbf{x}_{t_i} \mid \mathbf{y}) \left| \mathbb{E}_{p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i})} [f(\mathbf{x}_{t_{i-1}})] - \mathbb{E}_{q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}, \mathbf{x}_{t_i})} [f(\mathbf{x}_{t_{i-1}})] \right| \\ &\leq \frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} + \epsilon, \end{aligned} \quad (31)$$

where the last inequality follows from Lemma 1, and f can be assumed positive without loss of generality. By combining (30) and (31), we get

$$D_{\text{TV}}(p(\mathbf{x}_{t_{i-1}} \mid \mathbf{y}) \parallel q(\mathbf{x}_{t_{i-1}} \mid \mathbf{y})) \leq D_{\text{TV}}(p(\mathbf{x}_{t_i} \mid \mathbf{y}) \parallel q(\mathbf{x}_{t_i} \mid \mathbf{y})) + \frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} + \epsilon. \quad (32)$$

Applying this inequality recursively from $i = I$ down to $i = 1$, and noting that $t_I = T$ and $t_0 = 0$, we get

$$D_{\text{TV}}(p(\mathbf{x}_0 \mid \mathbf{y}) \parallel q(\mathbf{x}_0 \mid \mathbf{y})) \leq D_{\text{TV}}(p(\mathbf{x}_T \mid \mathbf{y}) \parallel q(\mathbf{x}_T \mid \mathbf{y})) + I \left(\frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} \right) + I\epsilon. \quad (33)$$

Since the initial state $\mathbf{x}_T$ is fixed (fully masked), we have $D_{\text{TV}}(p(\mathbf{x}_T | \mathbf{y}), q(\mathbf{x}_T | \mathbf{y})) = 0$. Therefore, we get

$$D_{\text{TV}}(q(\mathbf{x}_0 | \mathbf{y}) \parallel p(\mathbf{x}_0 | \mathbf{y})) \leq I \left(\frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} \right) + I\epsilon \quad (34)$$

Finally, note that $h(\epsilon, M, \lambda, D) = \tilde{\mathcal{O}}(1/\sqrt{M})$, so the overall bound is $\tilde{\mathcal{O}}(I/\sqrt{M} + I\epsilon)$. $\square$

Lemma 1. For any given query $\mathbf{y}$ and response $\mathbf{x}_{t_i}$, if

$$|\mathcal{E}(\mathbf{y}, \mathbf{x}_{t_i}) - \hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_{t_i})| \leq \delta \quad (35)$$

for $\delta = \epsilon$, then

$$\left| \mathbb{E}_{p(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] - \mathbb{E}_{q(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] \right| \leq \frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} + \epsilon \quad (36)$$

holds for any positive f with $\|f\|_\infty \leq 1$, where

$$h(\epsilon, M, \lambda, D) = \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{2}{M} \log \left(\frac{2}{\epsilon} \right)}. \quad (37)$$

Proof. Given $\mathbf{y}$ and $\mathbf{x}_{t_i}$, we know that

$$\mathbb{E}_{q(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] = \mathbb{E}_{\mathbf{x}_{t_{i-1}}} \left[\sum_{m=1}^M \frac{\hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))}{\sum_{m=1}^M \hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))} f(\mathbf{x}_{t_{i-1}}(m)) \right], \quad (38)$$

where $\mathbf{x}_{t_{i-1}}(m) \sim p_{\text{ref}}(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})$. Define the event

$$A_{t_i} = \left\{ \left| \frac{1}{M} \sum_{m=1}^M \mathcal{E}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m)) - C \right| \leq h(\epsilon, M, \lambda, D) \right\}. \quad (39)$$

By Lemma 2, we know that $\mathbb{P}(A_{t_i}^c) \leq \epsilon$. Then, by definition, we know

$$C - \epsilon - h(\epsilon, M, \lambda, D) \leq \frac{1}{M} \sum_{m=1}^M \hat{\mathcal{E}}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m)) \leq C + \epsilon + h(\epsilon, M, \lambda, D). \quad (40)$$

holds when event A_t happens. In this regime, by plugging this and (35) into (38), we get

$$\begin{aligned} & \mathbb{E}_{q(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] \\ &= \mathbb{E}_{q(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[(\mathbf{1}_{A_{t_i}} + \mathbf{1}_{A_{t_i}^c})f(\mathbf{x}_{t_{i-1}})] \\ &\leq \mathbb{E}_{\mathbf{x}_{t_{i-1}}} \left[\frac{1}{M} \sum_{m=1}^M \frac{\mathcal{E}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m)) + \epsilon}{C - \epsilon - h(\epsilon, M, \lambda, D)} f(\mathbf{x}_{t_{i-1}}(m)) \right] + \mathbb{E}_{q(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[\mathbf{1}_{A_{t_i}^c} f(\mathbf{x}_{t_{i-1}})] \\ &\leq \left(\frac{C}{C - \epsilon - h(\epsilon, M, \lambda, D)} \right) \mathbb{E}_{\mathbf{x}_{t_{i-1}}} \left[\frac{1}{M} \sum_{m=1}^M \frac{\mathcal{E}(\mathbf{y}, \mathbf{x}_{t_{i-1}}(m))}{C} f(\mathbf{x}_{t_{i-1}}(m)) \right] + \frac{\epsilon}{C - \epsilon - h(\epsilon, M, \lambda, D)} + \mathbb{P}(A_{t_i}^c) \\ &\leq \left(\frac{C}{C - \epsilon - h(\epsilon, M, \lambda, D)} \right) \mathbb{E}_{p(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] + \frac{\epsilon}{C - \epsilon - h(\epsilon, M, \lambda, D)} + \epsilon \\ &\leq \mathbb{E}_{p(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] + \frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C - \epsilon - h(\epsilon, M, \lambda, D)} + \epsilon. \end{aligned} \quad (41)$$

Similarly, we can prove the lower bound:

$$\mathbb{E}_{q(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] \geq \mathbb{E}_{p(\mathbf{x}_{t_{i-1}} | \mathbf{y}, \mathbf{x}_{t_i})}[f(\mathbf{x}_{t_{i-1}})] - \frac{2\epsilon + h(\epsilon, M, \lambda, D)}{C + \epsilon + h(\epsilon, M, \lambda, D)}. \quad (42)$$

Combining both bounds yields the desired result. $\square$

Lemma 2. For bounded $r(\mathbf{y}, \mathbf{x}_0) \in [-D, D]$, we have

$$\left| \frac{1}{M} \sum_{m=1}^M \mathcal{E}(\mathbf{y}, \mathbf{x}_s(m)) - C \right| \leq \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{2}{M} \log \left(\frac{2}{\delta} \right)} \quad (43)$$

holds with probability at least $1 - \delta$.

Proof. Notably, since $r(\mathbf{y}, \mathbf{x}_0) \in [-D, D]$, we know $\{\mathcal{E}(\mathbf{y}, \mathbf{x}_s(m))\}_{m=1}^M$ are M i.i.d. sub-Gaussian samples with coefficient $(e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}})^2/4$. Since $\mathbb{E}_{\mathbf{x}_s}[\mathcal{E}(\mathbf{y}, \mathbf{x}_s)] = C$, by Chernoff's inequality (Duchi, 2016), we prove our conclusion. $\square$

A.4. Proof of Importance Sampling

Proposition 4. For any given query $\mathbf{y}$, noisy response $\mathbf{x}_s$ and bounded $r(\mathbf{y}, \mathbf{x}_0) \in [-D, D]$, if there exists a constant $L > 0$ such that $l(\mathbf{x}_0) \leq L$ for all $\mathbf{x}_0$, then with probability at least $1 - \delta$ we have

$$\left| \mathcal{E}(\mathbf{y}, \mathbf{x}_s) - \hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_s) \right| \leq L e^{\frac{D}{\lambda}} \sqrt{\frac{\log(2/\delta)}{2K}}, \quad (12)$$

where

$$l(\mathbf{x}_0) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}{p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}. \quad (13)$$

Proof. Since $l(\mathbf{x}_0) \exp(r(\mathbf{y}, \mathbf{x}_0)/\lambda) \leq L e^{D/\lambda}$ for all $\mathbf{x}_0$, and $\{\mathbf{x}_0(k)\}_{k=1}^K$ are K i.i.d. samples from p_{small} , we can apply Hoeffding's inequality (Duchi, 2016) for bounded random variables:

$$\mathbb{P} \left(\left| \frac{1}{K} \sum_{k=1}^K l(\mathbf{x}_0(k)) - \mathbb{E}_{p_{\text{small}}}[l(\mathbf{x}_0)] \right| \geq \epsilon \right) \leq 2 \exp \left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}} \right). \quad (44)$$

Setting the right-hand side equal to δ and solving for ϵ gives:

$$\epsilon = L e^{\frac{D}{\lambda}} \sqrt{\frac{\log(2/\delta)}{2K}}. \quad (45)$$

Recalling that $\mathbb{E}_{p_{\text{small}}}[l(\mathbf{x}_0)] = \mathbb{E}_{p_{\text{ref}}}[\exp(r(\mathbf{y}, \mathbf{x}_0)/\lambda)] = \mathcal{E}(\mathbf{y}, \mathbf{x}_s)$, we prove our conclusion. $\square$

Theorem 1. For any given query $\mathbf{y}$, noisy response $\mathbf{x}_s$, bounded $r(\mathbf{y}, \mathbf{x}_0) \in [-D, D]$ and $\mathbf{x}_0 \sim q(\mathbf{x}_0 | \mathbf{y})$ as in Algorithm 2, if there exists a constant $L > 0$ such that $l(\mathbf{x}_0) \leq L$ for all $\mathbf{x}_0$, then we have

$$D_{\text{TV}}(q(\mathbf{x}_0 | \mathbf{y}) \| p(\mathbf{x}_0 | \mathbf{y})) \leq I \left(\frac{2\epsilon + g(\epsilon, M, K, L, \lambda, D)}{C - \epsilon - g(\epsilon, M, K, L, \lambda, D)} \right) + 4I \exp \left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}} \right) = \tilde{\mathcal{O}} \left(\frac{I}{\sqrt{M}} + \frac{I}{\sqrt{K}} \right), \quad (46)$$

where

$$\epsilon = \tilde{\mathcal{O}} \left(\frac{1}{\sqrt{K}} \right), \quad l(\mathbf{x}_0) = \frac{p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}{p_{\text{small}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)}, \quad g(\epsilon, M, K, L, \lambda, D) = \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{4K\epsilon^2}{ML^2 e^{\frac{2D}{\lambda}}}} = \tilde{\mathcal{O}} \left(\frac{1}{\sqrt{M}} \right). \quad (47)$$

Proof. For any given query $\mathbf{y}$ and noisy response $\mathbf{x}_s \in \{\mathbf{x}_{t_1}, \dots, \mathbf{x}_{t_I}\}$, let us define events

$$A_s = \left\{ \left| \mathcal{E}(\mathbf{y}, \mathbf{x}_s(m)) - \hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_s(m)) \right| \leq \epsilon \right\}, \quad B_s = \left\{ \left| \frac{1}{M} \sum_{m=1}^M \mathcal{E}(\mathbf{y}, \mathbf{x}_s(m)) - C \right| \leq h(\epsilon, M, K, L, \lambda, D) \right\}, \quad (48)$$

where

$$g(\epsilon, M, K, L, \lambda, D) = \left(\frac{e^{\frac{D}{\lambda}} - e^{-\frac{D}{\lambda}}}{2} \right) \sqrt{\frac{4K\epsilon^2}{ML^2 e^{\frac{2D}{\lambda}}}}, \quad (49)$$

By Proposition 4 and Lemma 2, we know that

$$\mathbb{P}(A_s^c) \leq 2 \exp \left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}} \right), \quad \mathbb{P}(B_s^c) \leq 2 \exp \left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}} \right). \quad (50)$$

Let $E_s = A_s \cap B_s$. We have

$$\mathbb{P}(E_s^c) \leq \mathbb{P}(A_s^c) + \mathbb{P}(B_s^c) \leq 4 \exp\left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}}\right) \quad (51)$$

When E_s occurs, we have

$$C - \epsilon - g(\epsilon, M, K, L, \lambda, D) \leq \frac{1}{M} \sum_{m=1}^M \hat{\mathcal{E}}_{\text{small}}(\mathbf{y}, \mathbf{x}_s(m)) \leq C + \epsilon + g(\epsilon, M, K, L, \lambda, D), \quad (52)$$

Following the same argument as in the proof of Proposition 3 and Lemma 1, we have

$$D_{\text{TV}}(q(\mathbf{x}_0 | \mathbf{y}) \parallel p(\mathbf{x}_0 | \mathbf{y})) \leq I \left(\frac{2\epsilon + g(\epsilon, M, K, L, \lambda, D)}{C - \epsilon - g(\epsilon, M, K, L, \lambda, D)} \right) + 4I \exp\left(-\frac{2K\epsilon^2}{L^2 e^{\frac{2D}{\lambda}}}\right). \quad (53)$$

Finally, note that $g(\epsilon, M, K, L, \lambda, D) = \tilde{\mathcal{O}}(\epsilon\sqrt{K/M})$, so we get $\tilde{\mathcal{O}}(I\epsilon(1 + \sqrt{K/M}) + I \exp(-K\epsilon^2))$. A common choice is to set $\epsilon = \tilde{\mathcal{O}}(1/\sqrt{K})$, so the overall bound is $\tilde{\mathcal{O}}(I/\sqrt{M} + I/\sqrt{K})$. $\square$

B. Implementation Details of ETS

B.1. Computational Complexity of ETS

To quantify our scaling, we analyze the total number of tokens processed by Algorithm 1. Let the guidance block size be $B = d_x/I$. The number of tokens generated over I steps is

$$\begin{aligned} N_{\text{tokens}} &= \sum_{i=1}^I M(B + K(d_x - iB)) \\ &= Md_x + IMKd_x - \frac{1}{2}(I+1)MKd_x \\ &= M \left(1 + \frac{I-1}{2}K \right) d_x. \end{aligned} \quad (54)$$

Thus, the latency of ETS is approximately N_{tokens}/d_x times that of a standard single-pass inference, which serves as a worst-case upper bound for both ARMs and DLMS. In practice, ARMs equipped with *batching* and *early-stop* mechanisms decode $\mathcal{O}(MK)$ sequences in parallel and often terminate early, yielding lower actual per-token cost and fewer generated tokens, as shown in Section 6.2.

B.2. Implementation Details of ETS Pipeline

Figure 6 illustrates our asynchronous evaluation pipeline designed for ARMs with dynamic generation times. For ARMs equipped with an *early-stop* mechanism, the generation time varies across different test examples, which would create idle periods (bubbles) in traditional synchronous evaluation pipelines (lm-eval). Inspired by the asynchronous rollout in AReaL (Fu et al., 2025), we designed this pipeline to fully utilize all of our acceleration techniques. The same pipeline can be applied to DLMS, although the potential for speedup is limited due to their relatively uniform generation times.

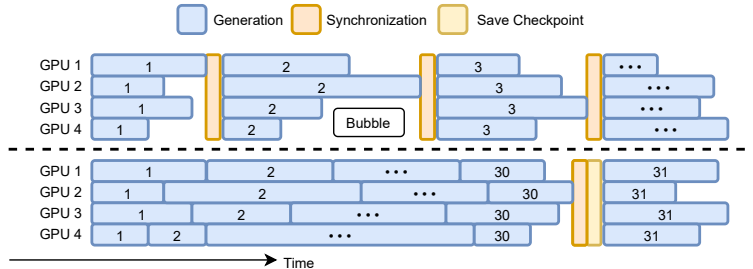


Figure 6. Asynchronous ETS evaluation pipeline for ARMs. Up: Traditional synchronous evaluation pipeline. Down: Asynchronous ETS evaluation pipeline. Numbers in the generation process denote the data index being evaluated on each GPU rank.

B.3. Implementation Details of Reward Calculating

Our reward is implemented via self-consistency: for a set of sampled completions, we first extract each completion’s final answer, identify the consensus answer by majority vote, and assign reward 1 to completions matching the consensus and 0 otherwise. To make this procedure more stable and efficient in practice, we adopt an answer caching method.

Concretely, during the entire ETS generation process, whenever we obtain a fully generated sample $\mathbf{x}_0$, we extract its final answer $a(\mathbf{x}_0)$ and append it to a global cache $\mathcal{A}_{\text{cache}}$.

Later, when estimating energies with self-consistency reward, we draw new completions $\{\mathbf{x}_0(i)\}_{i=1}^{MK}$, extract their answers $\{a(\mathbf{x}_0(i))\}_{i=1}^{MK}$, and compute the consensus answer over the union of cached and newly obtained answers. Then, we update the global cache $\mathcal{A}_{\text{cache}}$ with these newly obtained answers.

The answer caching method is reasonable because it simply increases the effective sample size used by self-consistency reward, without changing the underlying estimator. Theoretically, as long as the unique optimal answer has the highest probability mass among all possible answers, the probability that self-consistency selects an incorrect answer decays exponentially with the sample size.

Let A denote the extracted final answer from a single completion, taking values in a large discrete set $\mathcal{A}$. Let $a^* \in \mathcal{A}$ be the unique correct answer with probability

$$\mathbb{P}(A = a^*) = p, \quad \mathbb{P}(A = a) = q_a \quad (a \neq a^*), \quad \sum_{a \neq a^*} q_a = 1 - p. \quad (55)$$

Assume that $p > q_{\max}$ where $q_{\max} = \max_{a \neq a^*} q_a$. Given N i.i.d. samples $\{A_i\}_{i=1}^N$, self-consistency predicts the consensus answer

$$\hat{a} = \arg \max_{a \in \mathcal{A}} C_a, \quad C_a = \sum_{i=1}^N \mathbb{I}[A_i = a]. \quad (56)$$

Theorem 2 (Exponential decay of self-consistency voting error). *Under the assumption $p > q_{\max}$ and i.i.d. sampling, the probability that self-consistency selects an incorrect answer satisfies*

$$\mathbb{P}(\hat{a} \neq a^*) \leq \sum_{a \neq a^*} \exp\left(-\frac{N(p - q_a)^2}{2}\right) \leq (|\mathcal{A}| - 1) \exp\left(-\frac{N(p - q_{\max})^2}{2}\right), \quad (57)$$

where $|\mathcal{A}|$ can be replaced by the number of answers with nonzero probability mass if $\mathcal{A}$ is large.

Proof. Self-consistency makes an error if and only if there exists some $a \neq a^*$ whose vote count is no smaller than that of a^* , i.e.,

$$\{\hat{a} \neq a^*\} \subseteq \bigcup_{a \neq a^*} \{C_a \geq C_{a^*}\}. \quad (58)$$

By the union bound,

$$\mathbb{P}(\hat{a} \neq a^*) \leq \sum_{a \neq a^*} \mathbb{P}(C_a \geq C_{a^*}). \quad (59)$$

Fix any $a \neq a^*$ and define for each sample

$$Y_i^{(a)} = \mathbb{I}[A_i = a^*] - \mathbb{I}[A_i = a], \quad Y_i^{(a)} \in \{-1, 0, 1\} \subset [-1, 1]. \quad (60)$$

Then

$$\sum_{i=1}^N Y_i^{(a)} = C_{a^*} - C_a, \quad \mathbb{E}[Y_i^{(a)}] = p - q_a > 0. \quad (61)$$

Therefore,

$$\mathbb{P}(C_a \geq C_{a^*}) = \mathbb{P}\left(\sum_{i=1}^N Y_i^{(a)} \leq 0\right) = \mathbb{P}\left(\frac{1}{N} \sum_{i=1}^N Y_i^{(a)} - (p - q_a) \leq -(p - q_a)\right). \quad (62)$$

Applying Hoeffding’s inequality for independent bounded random variables in $[-1, 1]$ yields

$$\mathbb{P}(C_a \geq C_{a^*}) \leq \exp\left(-\frac{N(p - q_a)^2}{2}\right). \quad (63)$$

Summing over all $a \neq a^*$ proves the first inequality. The second inequality follows from $p - q_a \geq p - q_{\max}$ for all $a \neq a^*$. $\square$

B.4. The Candidates of Reward Designs

Logits. The log-probability reward measures how likely a sampled response trajectory is under the reference policy. Intuitively, responses with higher average token log-probability lie in higher-density regions of $p_{\text{ref}}(x_0 | y)$, and thus reflect stronger model confidence in its own generation. We define the reward as the average per-token log-probability along the trajectory:

$$r_{\text{logp}}(\mathbf{y}, \mathbf{x}_0) = \frac{1}{d_x} \sum_{i=1}^{d_x} \log p_{\text{ref}}(x_0^i | \mathbf{y}, \mathbf{x}_0^{<i}). \quad (64)$$

Entropy. The entropy reward captures predictive uncertainty at each generation step by inspecting the next-token distribution $p_{\text{ref}}(\cdot | \mathbf{y}, \mathbf{x}_0^{<i})$. A more concentrated distribution indicates higher certainty, while a flatter distribution indicates higher uncertainty. To align the direction of optimization with confidence, we use the *negative entropy* and average it over time:

$$r_{\text{ent}}(\mathbf{y}, \mathbf{x}_0) = \frac{1}{d_x} \sum_{i=1}^{d_x} \sum_{v \in V} p_{\text{ref}}(v | \mathbf{y}, \mathbf{x}_0^{<i}) \log p_{\text{ref}}(v | \mathbf{y}, \mathbf{x}_0^{<i}). \quad (65)$$

Self-Certainty. Self-certainty quantifies how far the model’s next-token distribution deviates from the uniform distribution. When the predictive distribution is strongly peaked, it departs more from uniformity, indicating higher confidence. Following the standard definition, we compute an average token-level score based on the log-likelihood ratio against a uniform baseline:

$$r_{\text{sc}}(\mathbf{y}, \mathbf{x}_0) = -\frac{1}{d_x |V|} \sum_{i=1}^{d_x} \sum_{v \in V} \log(|V| \cdot p_{\text{ref}}(v | \mathbf{y}, \mathbf{x}_0^{<i})). \quad (66)$$

Up to an additive constant, this is equivalent to the divergence $D_{\text{KL}}(U \| p_{\text{ref}}(\cdot | \mathbf{y}, \mathbf{x}_0^{<i}))$, where U denotes the uniform distribution over V .

Self-Consistency. Self-consistency uses agreement among multiple independent samples as a proxy for reliability. Given the same conditioning input, we sample K i.i.d. responses $\{\mathbf{x}_0(k)\}_{k=1}^K$ from $p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y})$ (or from $p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}, \mathbf{x}_s)$ when conditioning on an intermediate state $\mathbf{x}_s$), and extract a discrete final answer $a(k)$ from each completion (e.g., via a task-specific parsing function). We then define the consensus answer by majority voting and reward samples that match the consensus:

$$\begin{aligned} \mathbf{x}_0(k) &\stackrel{\text{i.i.d.}}{\sim} p_{\text{ref}}(\mathbf{x}_0 | \mathbf{y}), \quad k = 1, \dots, K, \\ a(k) &= \text{Extract}(\mathbf{x}_0(k)), \quad a^* = \text{mode}(\{a(k)\}_{k=1}^K), \\ r_{\text{cons}}(\mathbf{y}, \mathbf{x}_0(k)) &= \mathbb{I}[a(k) = a^*]. \end{aligned} \quad (67)$$

C. Implementation Details of Experiments

C.1. Baseline Settings

For the Base method, we follow the original settings from (Yang et al., 2025) and (Nie et al., 2025).

Best-of-N is naturally integrated into our ETS framework as a special case, with detailed hyperparameters provided in Appendix C.2.

For Beam Search, we use the standard implementation in the transformers (Wolf et al., 2020) to evaluate ARMs with original temperature $t = 0.7$ (refer to Appendix D.3), leveraging its parallel decoding via batching. For DLMs, we implement beam search ourselves; however, due to their iterative generation nature, DLMs cannot be accelerated via batching in the same way as ARMs.

For Power Sampling (Karan & Du, 2025), we retain the original $\alpha = 0.25$, $N_{\text{MCMC}} = 10$ and $B = d_x/16$ settings, but reduce the generation length d_x from 3072 to 512 due to its *substantial* latency of $(N_{\text{MCMC}} d_x)/(4B) = 40\times$, which places it in a different computational regime compared to other TTS methods. Furthermore, unlike general evaluation frameworks such as lm-eval, Power Sampling requires a hand-crafted, task-specific verifier, which precludes its evaluation on GSM8K.

For the GRPO baseline, we post-trained Qwen3-1.7B and Qwen3-8B in the non-thinking mode on the MATH dataset for 205 training steps, with greedy decoding for evaluation. For DLMs, we directly use the results from LLaDA-1.5 fine-tuned with VRPO.

C.2. Hyperparameter Settings of ETS

Table 2, 3 and 4 list the hyperparameters used in the experiments summarized in Table 1. For ARMs, we configure the generation length $d_x = 512$ (ablation study in Appendix D.4). For DLMs, we retain the original dynamic generation lengths, which vary across datasets. The selection of hyperparameters follows the findings from the ablation study in Section 6.3 and Appendix D. For the KL regularization coefficient λ , we fix it to 0.1.

Notably, ETS-IS achieves comparable or even superior accuracy-latency trade-offs to the original ETS under certain hyperparameter settings. Our hyperparameter selection thus proceeds in two stages: we first identify a strong ETS configuration, then adjust it with reference to the latency achieved by Best-of-N while preserving accuracy. For ARMs, we employ Qwen3-1.7B as p_{small} to accelerate energy estimation for Qwen3-8B. We do not evaluate ETS-IS on Qwen3-1.7B, because its base inference speed is already sufficiently fast and the extra overhead of computing logits for importance sampling would outweigh any acceleration benefit. For DLMs, the introduction of Fast-dLLM provides substantial acceleration, so we directly adopt the ETS hyperparameters.

Table 2. Hyperparameter settings in Table 1 for Qwen3-1.7B model. t is the *softmax temperature* for LLM generation. N_{tokens}/d_x provides an upper bound on the *multiplicative latency* relative to a standard single-pass (base) inference, which has $N_{\text{tokens}}/d_x = 1$.

Methods	Benchmarks	M	K	I	B	t	N_{tokens}/d_x	Time
Beam Search	MATH500	20	-	-	-	0.7	20	$1.47\times$
	GSM8K	20	-	-	-	0.7	20	$1.63\times$
	GPQA	20	-	-	-	0.7	20	$1.89\times$
	HumanEval	20	-	-	-	0.7	20	$1.24\times$
Best-of-N	MATH500	50	1	1	512	0.7	50	$4.73\times$
	GSM8K	20	1	1	512	1.5	20	$2.97\times$
	GPQA	30	1	1	512	0.5	30	$3.87\times$
	HumanEval	50	1	1	512	0.25	50	$3.30\times$
ETS	MATH500	15	3	8	64	0.7	172.5	$8.94\times$
	GSM8K	15	3	8	64	1.5	172.5	$6.92\times$
	GPQA	5	3	8	64	1.5	57.5	$5.94\times$
	HumanEval	10	10	8	64	0.25	360	$5.53\times$

Table 3. Hyperparameter settings in Table 1 for Qwen3-8B model. t is the *softmax temperature* for LLM generation. N_{tokens}/d_x provides an upper bound on the *multiplicative latency* relative to a standard single-pass (base) inference, which has $N_{\text{tokens}}/d_x = 1$.

Methods	Benchmarks	M	K	I	B	t	N_{tokens}/d_x	Time
Beam Search	MATH500	20	-	-	-	0.7	20	$1.57\times$
	GSM8K	20	-	-	-	0.7	20	$1.88\times$
	GPQA	50	-	-	-	0.7	50	$4.48\times$
	HumanEval	20	-	-	-	0.7	20	$1.18\times$
Best-of-N	MATH500	50	1	1	512	0.7	50	$6.09\times$
	GSM8K	30	1	1	512	1.5	30	$4.89\times$
	GPQA	30	1	1	512	1.5	30	$5.48\times$
	HumanEval	50	1	1	512	0.25	50	$4.09\times$
ETS	MATH500	10	3	8	64	0.7	115	$9.85\times$
	GSM8K	15	3	4	128	1.5	82.5	$6.21\times$
	GPQA	15	3	8	64	0.25	172.5	$5.71\times$
	HumanEval	20	3	8	64	0.25	230	$4.67\times$
ETS-IS	MATH500	15	3	4	128	0.7	82.5	$5.88\times$
	GSM8K	15	3	4	128	1.5	82.5	$5.33\times$
	GPQA	15	3	8	64	0.25	172.5	$5.50\times$
	HumanEval	10	3	8	64	0.25	115	$2.55\times$

Table 4. Hyperparameter settings in Table 1 for LLaDA-8B-Instruct model. d_x is the answer length that LLaDA is allowed to generate for the final output sequence. Block length denotes the number of tokens generated per block under LLaDA’s semi-autoregressive strategy. t is the *softmax temperature* for LLM generation. N_{tokens}/d_x provides an upper bound on the *multiplicative latency* relative to a standard single-pass (base) inference, which has $N_{\text{tokens}}/d_x = 1$.

Methods	Benchmarks	d_x	Block length	M	K	I	B	t	N_{tokens}/d_x	Time
Beam Search	MATH500	256	32	20	-	-	-	0.5	20	9.38×
	GSM8K	256	8	20	-	-	-	1.0	20	22.5×
	GPQA	64	8	20	-	-	-	0.5	20	13.9×
	HumanEval	512	32	20	-	-	-	0.25	20	10.5×
Best-of-N	MATH500	256	32	20	1	1	256	0.5	20	14.9×
	GSM8K	256	8	15	1	1	256	1.0	15	14.7×
	GPQA	64	8	20	1	1	128	0.5	20	22.9×
	HumanEval	512	32	20	1	1	512	0.25	20	24.5×
ETS	MATH500	256	32	15	1	4	64	0.5	37.5	20.6×
	GSM8K	256	8	10	1	4	64	1.0	25	30.9×
	GPQA	64	8	5	3	8	16	0.5	57.5	57.1×
	HumanEval	512	32	5	3	8	64	0.25	57.5	47.7×
ETS-IS	MATH500	256	32	5	3	8	32	0.5	57.5	7.24×
	GSM8K	256	8	10	1	4	64	1.0	25	17.2×
	GPQA	64	8	5	3	8	16	0.5	57.5	19.1×
	HumanEval	512	32	5	3	8	64	0.25	57.5	21.6×

D. Additional Experimental Results

D.1. More Results on DLMs

Following the results in Section 6.3, we conduct similar ablation studies on DLMs, including the variations over $M \times K$, guidance step I , and accuracies under various latencies. For time considerations, we use 200 randomly selected examples from the GSM8K dataset for this ablation on LLaDA.

As shown in Figure 7 and Figure 8, we can obtain the same conclusions as in Section 6.3 on ARMs. Specifically, an appropriate choice of $M \times K$ can improve performance while maintaining low latency. Both accuracy and latency increase as $M \times K$ increases, and scaling M is empirically more effective than scaling K . For the guidance schedule, increasing the number of guide steps I strengthens guidance but incurs a higher latency. Therefore, we recommend selecting I to balance accuracy and computational efficiency. For latency and accuracy, ETS achieves higher accuracy at comparable latency than other TTS baselines, and its advantage becomes more pronounced as the computational budget increases.

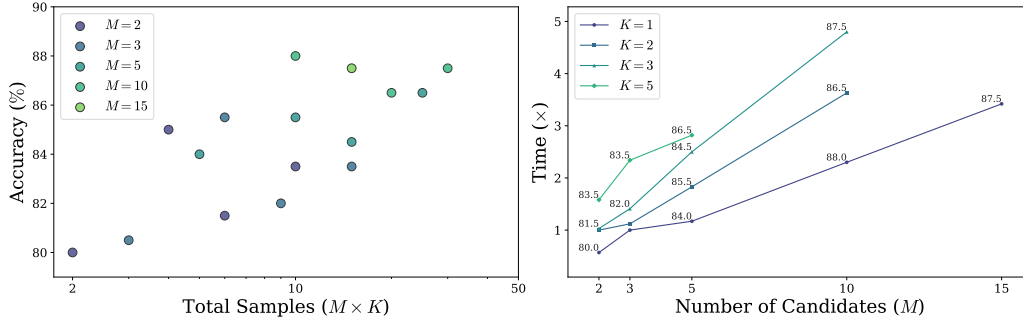


Figure 7. Effect of total samples on ETS. We ablate the hyperparameter settings with LLaDA-8B-Instruct and plot split GSM8K accuracies (left) with corresponding latencies (right) for various sample counts (the accuracies are also reported).

D.2. Reward Design

In Figure 9, we compare the correlations between the ground-truth reward and four self-evaluation reward candidates on GSM8K under Qwen3-1.7B model. As can be seen, only the self-consistency reward used in our ETS exhibits a clear separation between the two classes: answers that agree with the consensus receive consistently high reward, whereas incorrect answers receive low reward. In contrast, the distributions of the other three rewards largely overlap for correct and incorrect responses, meaning their numerical values do not reliably indicate semantic accuracy. This result confirms that

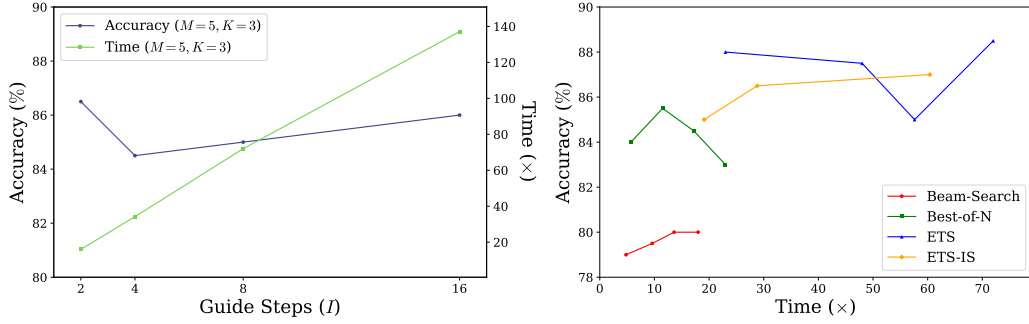


Figure 8. Left: Effect of guidance steps on ETS. We evaluate LLaDA-8B-Instruct on splitted GSM8K with corresponding latencies are reported under various guidance steps. Right: Comparisons between TTS methods. We ablate the latencies under the LLaDA-8B-Instruct models evaluated on splitted GSM8K for various TTS methods.

voting is uniquely effective as a self-evaluation signal for energy estimation in reasoning tasks.

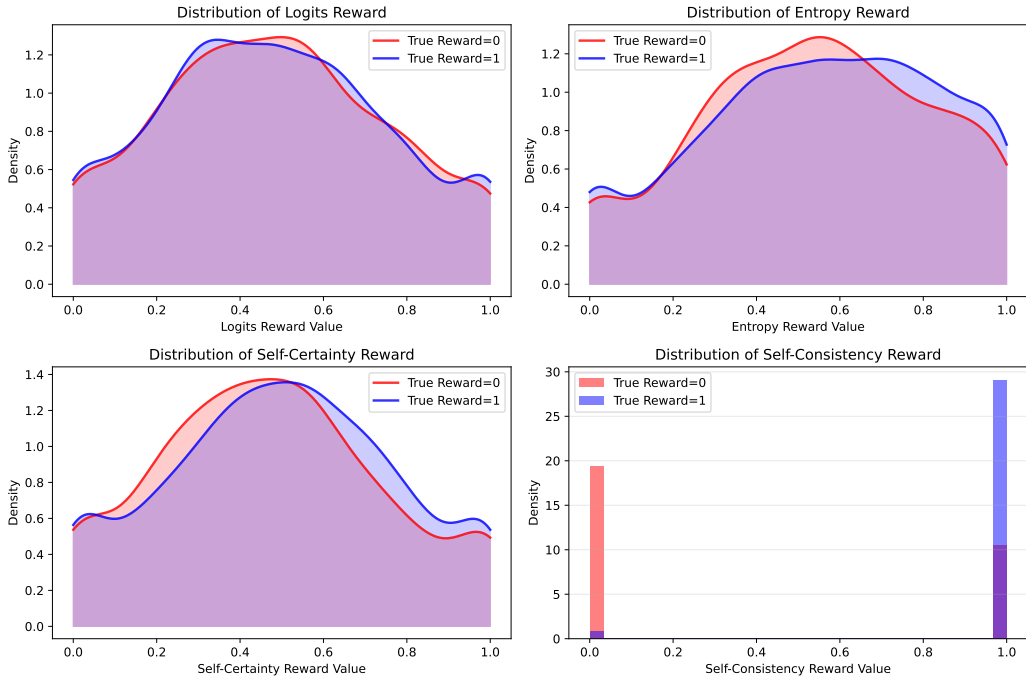


Figure 9. Comparison of self-evaluation rewards. We plot distributions of four rewards evaluated on GSM8K with Qwen3-1.7B (60% base accuracy), stratified by ground-truth reward (red: incorrect, blue: correct).

To further quantify the effectiveness of the self-consistency reward, Table 5 reports the performance of Qwen3-8B on GSM8K using both the self-consistency reward and the ground-truth reward. While the ground-truth reward naturally yields the best performance (e.g., pass@N accuracy of 98.10 for Best-of-N), our self-consistency reward achieves competitive results (94.09 for Best-of-N and 94.22 for ETS). This demonstrates that self-consistency serves as a practical, training-free proxy that closely approximates the true reward signal, validating its use in ETS when the ground-truth reward is unavailable during inference.

D.3. Temperature

Following the settings in Appendix C.2, the temperature t is adjusted based on the dataset and model characteristics: a lower temperature constrains sampling variability for models with lower base accuracy (or those that are only pre-trained), while a higher temperature promotes broader exploration for models with higher base accuracy (or post-trained models). Furthermore, a lower temperature encourages ARMs to early-stop more deterministically, thereby reducing the latency of

Table 5. Effect of reward on ETS. We evaluate Qwen3-8B on GSM8K under various rewards.

Reward	Best-of-N	ETS	ETS-IS
Logits	88.48	88.70	88.40
Entropy	90.30	89.54	88.93
Self-Certainty	90.07	89.46	88.93
Self-Consistency	94.09	94.22	91.81
Ground-Truth	98.10 (pass@N)	97.57	95.00

ETS. However, a single temperature may not be optimal across all hyperparameter scales.

A notable exception is the GPQA dataset, where the verifier simply extracts the first capital letter from the response. As illustrated in Figure 10, this narrow criterion does not diminish the benefits of temperature tuning at high temperatures; instead, it leads to comparable performance between $t = 0.1$ and $t = 1.5$, indicating that even high temperatures can yield gains in this setting.

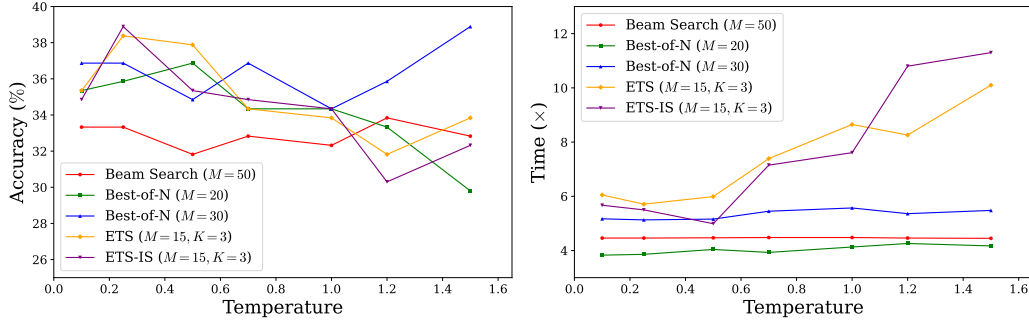


Figure 10. Effect of temperature on ETS. We ablate the temperature on Qwen3-8B and plot GPQA accuracies (left) with corresponding latencies (right).

Empirically, the optimal temperature is shared between Best-of-N and ETS with comparable latency (Chow et al., 2024), while Beam Search is insensitive to temperature (so we fix $t = 0.7$). Based on this, extensive experimental evidence suggests exploring t within the range of $[0.25, 1.5]$.

D.4. Generation Length

We ablate the maximum generation length d_x for reasoning sequences. Results in Table 6 show that increasing d_x beyond 512 tokens yields diminishing returns on most benchmarks (e.g., HumanEval). MATH500 is an exception, where longer sequences ($d_x = 1024$) are beneficial, due to their more complex reasoning chains. Based on this efficiency trade-off, we fix $d_x = 512$ for all main experiments on ARMs. For DLMs, we follow the original settings of LLaDA (in Table 4).

Table 6. Performance across generation lengths. We ablate the d_x on Qwen3-8B and bold the best accuracy value for each method across different generation lengths.

Length	MATH500						HumanEval					
	Base	Best-of-N	Beam Search	ETS	ETS-IS	GRPO	Base	Best-of-N	Beam Search	ETS	ETS-IS	GRPO
256	47.2	49.8	44.6	55.6	49.4	56.0	60.98	67.07	67.68	68.90	62.80	62.80
512	60.0	65.2	61.2	72.4	66.2	69.2	58.54	67.07	69.51	71.34	68.9	65.85
1024	67.2	76.2	69.8	76.8	75.0	77.0	61.59	62.20	68.29	67.07	60.37	65.85

D.5. Long-CoT Task

To address the concern that intrinsic self-consistency may be noisy in hard tasks, we have extended our evaluation to AIME24 (Zhang & Math-AI, 2024) on Qwen3-8B, a challenging long-CoT benchmark. Using the same reward design, we observe consistent improvements in accuracy, confirming that our method remains effective even when reasoning chains are long (6144 tokens) and diverse.

Furthermore, for long-CoT tasks, the high cost of p_{ref} limits the sample budget of standard TTS methods. In contrast, ETS-IS achieves a large gain over ETS by exploiting p_{small} to explore many more total samples within a comparable latency budget. This greater exploration yields substantially higher accuracy, as shown in Table 7.

Table 7. Performance on the AIME24 benchmark with Qwen3-8B (average over 32 samples), with best accuracy bolded.

Method	AIME24 (avg@32)	Time (8×GPU)
Base	16.25	1×
Beam Search	16.46	1.64×
Best-of-N	18.23	3.34×
ETS	18.54	4.03×
ETS-IS	22.40	2.82×
GRPO	21.57	1×

D.6. ETS Asynchronous Pipeline

To fully exploit the parallel nature of ETS, we implement an asynchronous evaluation pipeline (Appendix B.2) that continuously dispatches generation tasks to GPU ranks, eliminating idle waiting periods common in synchronous evaluation loops (e.g., lm-eval).

As shown in Table 8, the pipeline yields different speedups depending on the inherent parallelism of each method. ETS and ETS-IS naturally generate many candidates and completions in parallel, allowing them to saturate the GPUs more effectively and thus obtain a larger relative speedup. In contrast, methods with lower parallelism (e.g., Base, Best-of-N) gain less acceleration. This asymmetry reflects algorithmic characteristics rather than unfairness. For a completely orthogonal assessment of algorithmic efficiency, we also report synchronous runtimes as a supplementary reference.

Table 8. Synchronous vs. asynchronous runtime on MATH500 with Qwen3-8B. The asynchronous pipeline is applied uniformly to all methods. Gains are computed as $(1 - \text{Async}/\text{Sync}) \times 100\%$.

Method	Number of x_0	Time (8×GPU)		Gain
		Sync	Async	
Base	1	0:15:30	0:15:07	2.5%
Best-of-N	50	0:40:23	0:37:26	7.5%
ETS	115	1:32:05	1:02:32	32.6%
ETS-IS	82.5	1:00:12	0:43:35	27.6%

Accuracy remains unchanged between synchronous and asynchronous execution for all methods, confirming that the pipeline does not affect generation quality. The reported main results (Table 1) use the asynchronous pipeline for all TTS methods.

E. Generation Example of ETS

Table 9 presents a step-by-step generation example of the ETS method applied to a GSM8K math problem by LLaDA-8B-Instruct. It illustrates the intermediate candidate completions, their corresponding energy scores, and the selection process across multiple guidance steps, culminating in the final answer. For every guidance step (1–6), the model produces five candidates (Candidate Index 1–5). For each candidate, a Monte Carlo procedure is applied to obtain three (x_0) samples, from which final answers are extracted (shown in the “Extracted Answer” column). Taking guidance step 1 as an example, Candidate 1 yields the extracted answers (['\$90,0', '\$75,000', '\$90,000']), while Candidate 2 yields (['70000', '70000', '20000']), and so on for the remaining candidates. Within a given guidance step, the consensus answer is treated as the step-level target; here the consensus is '70000'. Rewards are then assigned by comparing each extracted answer to the consensus: answers matching the consensus receive reward 1, and mismatches receive reward 0. These rewards are aggregated into an energy that reflects the candidate’s relative preference under the ETS selection rule. Consequently, Candidate 1 in Step 1 receives rewards ([0,0,0]) and has Energy (=1), whereas Candidate 2 receives rewards ([1,1,0]) and attains a substantially stronger selection score (Energy (=14684.64) as reported). The “Selected Candidate” column records the candidate chosen for that step (Candidate 2 for Step 1), and the same consensus–reward–energy–selection mechanism is repeated identically for subsequent steps until the final solution is formed, culminating in the stated final answer 70000.

Table 9. Step-by-step generation example of ETS on GSM8K using LLaDA-8B-Instruct. The table illustrates the intermediate candidate texts, their energy scores, and the selection process across multiple guidance steps ($I = 8$) for a math word problem. The response’s meaningful content is completed at guidance step 6, and subsequent steps produce only the `<|endoftext|>` token, which is omitted for space consideration. (Part 1 of 2)

Question: Josh decides to try flipping a house. He buys a house for \$80,000 and then puts in \$50,000 in repairs. This increased the value of the house by 150%. How much profit did he make?\nAnswer:					
Step	Candidate Index	Generated Text	Extracted Answer	Energy	Selected Candidate
1	1	To determine Josh’s profit, we need to follow these steps:\n\n1. Calculate the increased value of the house after the repairs.\n2. Calculate	['\$90,0', '\$75,000', '\$90,000']	1	2
	2	Josh bought the house for \$80,000 and put \$50,000 in repairs, so his total cost was \$80	['70000', '70000', '20000']	14684.64	
	3	To determine Josh’s profit, we need to follow these steps:\n\n1. Calculate the increased value of the house after the repairs.\n2. Sub	['120,000', '45,000', '\$120, 000']	1	
	4	To determine the profit Josh made, we need to follow these steps:\n\n1. Calculate the increased value of the house after the repairs.\n2.	['120,000', '45,000', '\$120,000']	1	
	5	Josh bought the house for \$80,000 and spent \$50,000 on repairs, so his total cost was \$80	['5000', '195000', '70000']	7342.82	
2	1	,000 + \$50,000 = \$130,000.\nThe repairs increased the value of the house by	['70000', '70000', '70000']	22026.47	1
	2	,000 + \$50,000 = \$130,000.\nThe repairs increased the value of the house by	['10000', '70000', '65000']	7342.82	
	3	,000 + \$50,000 = \$130,000.\nThe repairs increased the value of the house by	['70000', '70000', '70000']	22026.47	
	4	,000 + \$50,000 = \$130,000.\nHe sold the house for a value that increased	['10000', '195000', '195000']	1	
	5	,000 + \$50,000 = \$130,000.\nThe repairs increased the value of the house by	['70,000', '70000', '5000']	7342.82	
3	1	150%, so the increased value is 150% of \$80,000, which is 1.5 * \$	['70000', '\$70,000', '70000']	14684.64	4
	2	150%, so the new value of the house was \$80,000 * 1.5 = \$120,0	['10000', '20000', '10000']	1	
	3	150%, so the new value of the house is \$80,000 * 1.5 = \$120,0	['100000', '2,000', '50000']	1	
	4	150%, so the house is now worth \$80,000 + (150/100) * \$80	['70000', '70000', '70000']	22026.47	
	5	150%, so the new value of the house is \$80,000 * 150/100 = \$1	['10000', '70000', '5000']	7342.82	

Table 9. (Continued) Step-by-step generation example of ETS on GSM8K using LLaDA-8B-Instruct. (Part 2 of 2)

Step	Candidate Index	Generated Text	Extracted Answer	Energy	Selected Candidate
4	1	,000 = \$80,000 + \$120,000 = \$200,000.\n	['70000', '70000', '70000']	22026.47	5
	2	,000 = \$80,000 + \$120,000 = \$200,000.\n	['70000', '70000', '70000']	22026.47	
	3	,000 = \$80,000 + \$120,000 = \$200,000.\n	['70000', '70000', '70000']	22026.47	
	4	,000 = \$80,000 + \$120,000 = \$200,000.\n	['70000', '70000', '70000']	22026.47	
	5	,000 = \$80,000 + \$120,000 = \$200,000.\n	['70000', '70000', '70000']	22026.47	
5	1	He sold the house for \$200,000, so his profit is \$200,000 - \$130	['70000', '70000', '70000']	22026.47	4
	2	To find the profit, we subtract the total cost from the new value of the house: \$200,000 - \$130,	['70000', '70000', '70000']	22026.47	
	3	To find the profit, we subtract his total cost from the new value of the house: \$200,000 - \$130,	['70000', '70000', '70000']	22026.47	
	4	To find the profit, we subtract the total cost from the new value of the house: \$200,000 - \$130,	['70000', '70000', '70000']	22026.47	
	5	To find the profit, we subtract the total cost from the new value of the house: \$200,000 - \$130,	['70000', '70000', '70000']	22026.47	
6	1	,000 = \$70,000.\n#### 70000	['70000', '70000', '70000']	22026.47	2
	2	,000 = \$70,000.\n#### 70000	['70000', '70000', '70000']	22026.47	
	3	,000 = \$70,000.\n#### 70000	['70000', '70000', '70000']	22026.47	
	4	,000 = \$70,000.\n#### 70000	['70000', '70000', '70000']	22026.47	
	5	,000 = \$70,000.\n#### 70000	['70000', '70000', '70000']	22026.47	
Final Answer: Josh bought the house for \$80,000 and put \$50,000 in repairs, so his total cost was \$80,000 + \$50,000 = \$130,000.\nThe repairs increased the value of the house by 150%, so the house is now worth \$80,000 + (150/100) * \$80,000 = \$80,000 + \$120,000 = \$200,000.\nHe sold the house for \$200,000, so his profit is \$200,000 - \$130,000 = \$70,000.\n#### 70000					